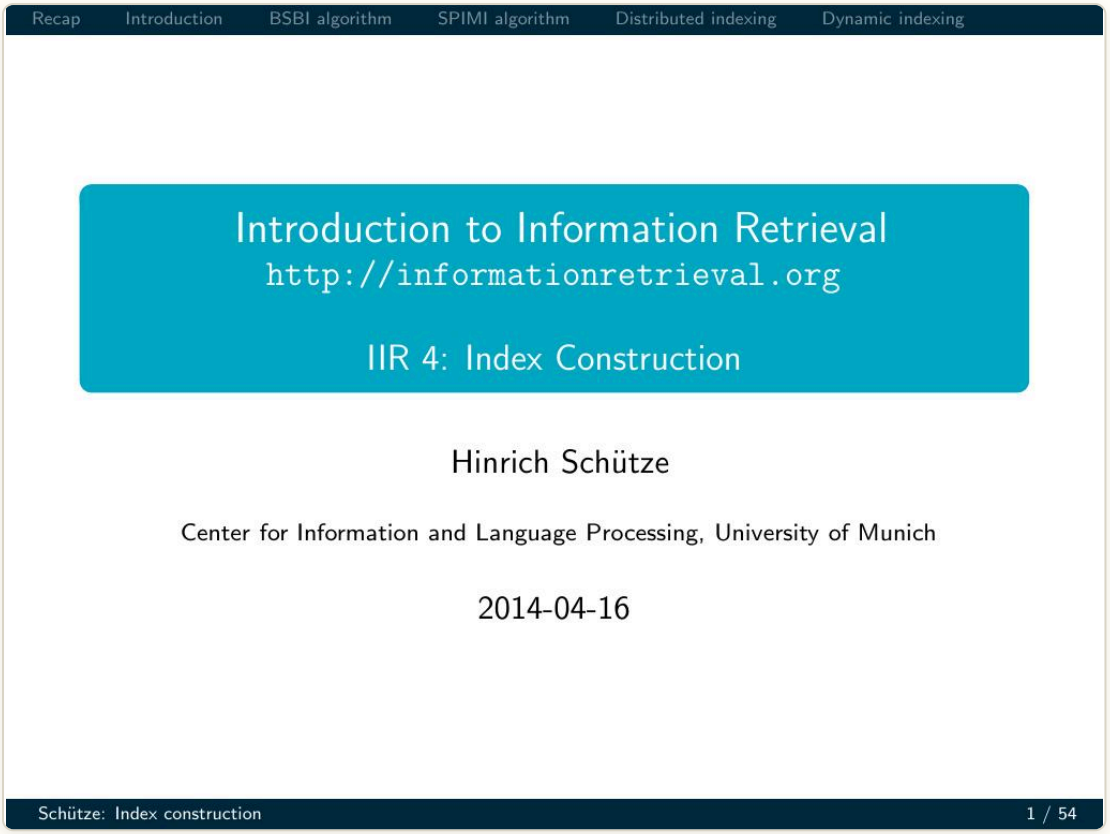
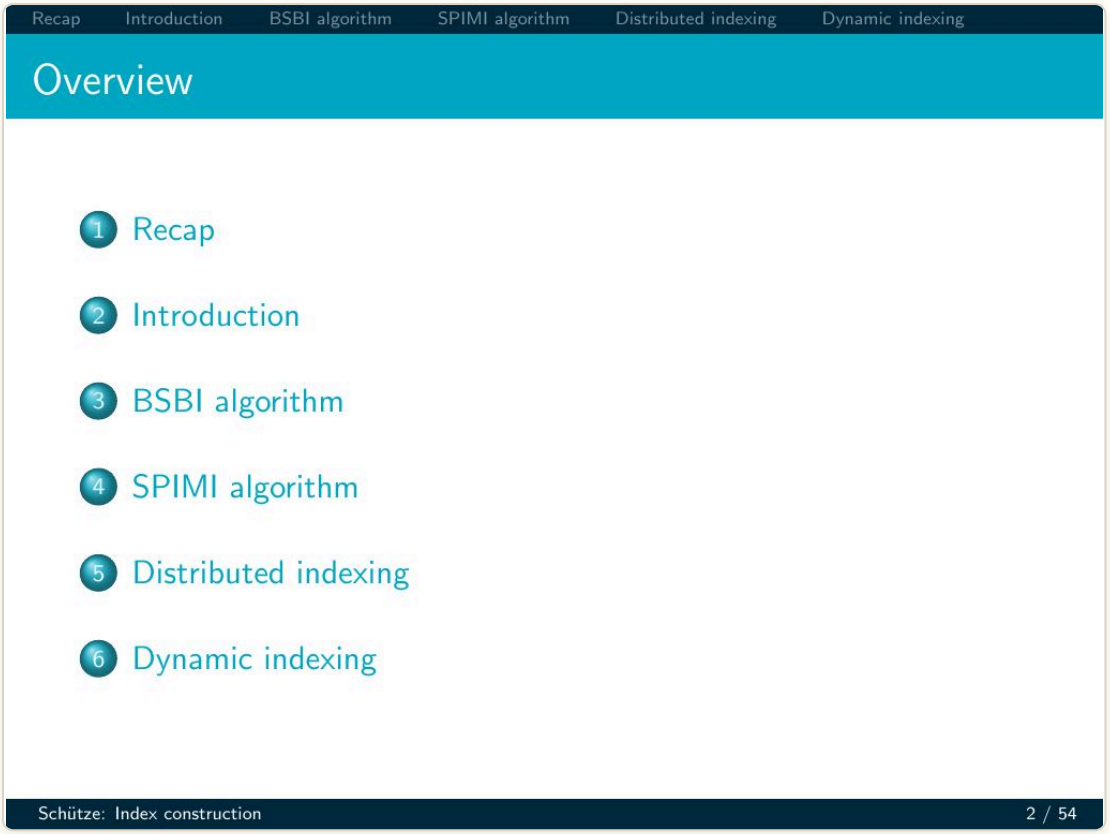


정보검색 입문 — IIR 4: 색인 구축

- Hinrich Schütze, 뮌헨대 (2014-04-16)

✍️ 메모

💡 이해 4강 핵심: 하드웨어 제약 → BSBI(정렬 기반 블록) → SPIMI(해시·정렬 안 함) → MapReduce 분산 → 동적 색인(보조 색인, 로그 병합)

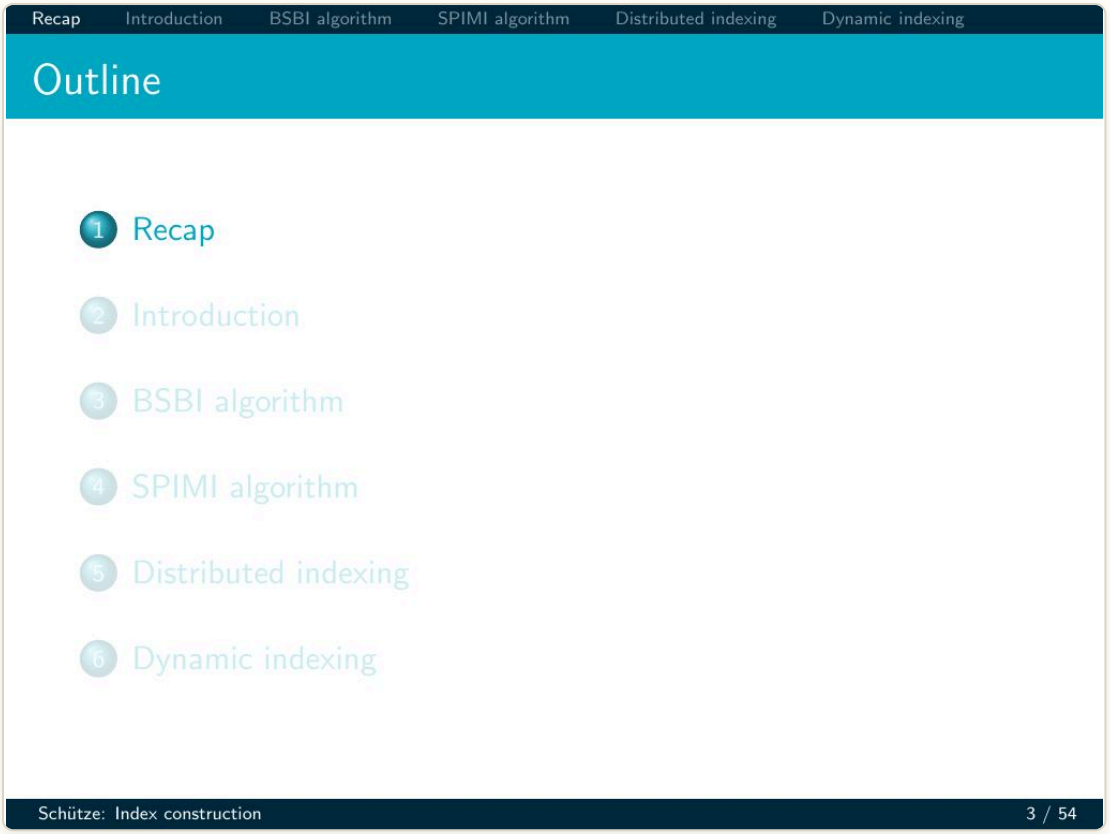


개요

- ① 복습 ② 소개 ③ BSBI 알고리즘 ④ SPIMI 알고리즘 ⑤ 분산 색인 ⑥ 동적 색인

✍ 메모

— 넘어가도 되는 쪽

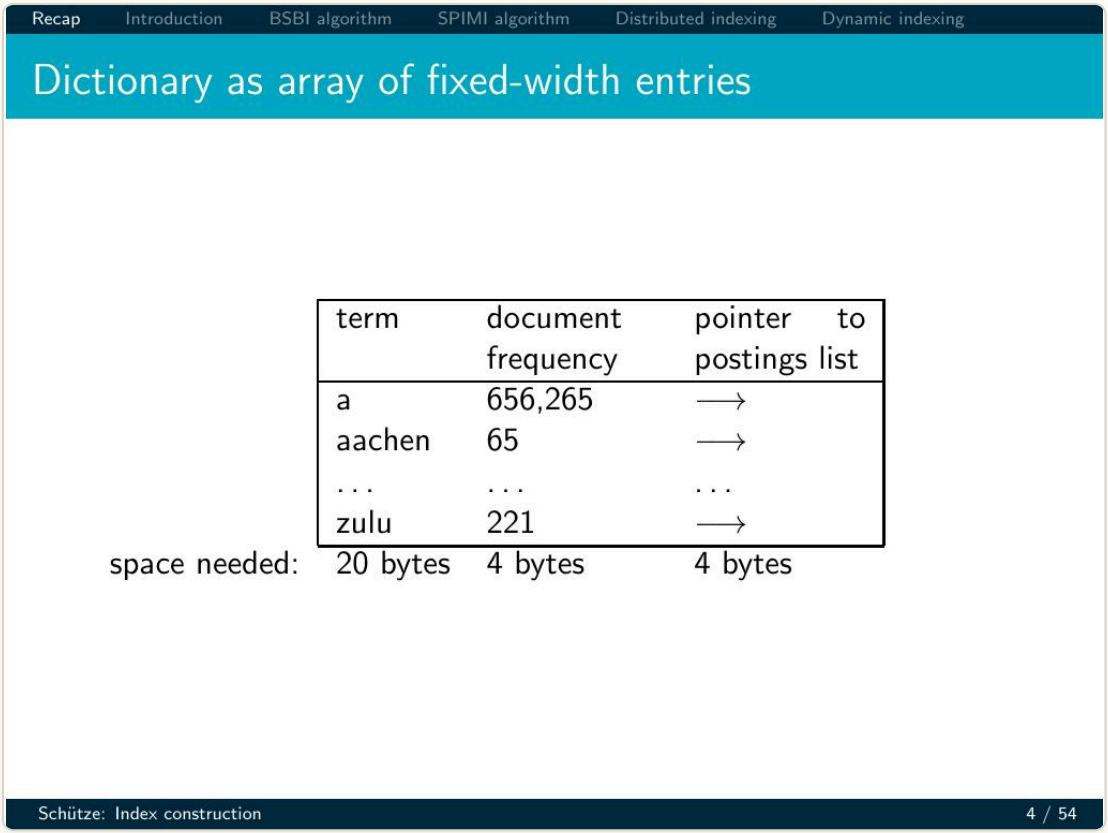


목차

- 복습 → 소개(하드웨어) → BSBI → SPIMI → 분산 색인 → 동적 색인
- 지금: 복습

✍ 메모

— 넘어가도 되는 쪽

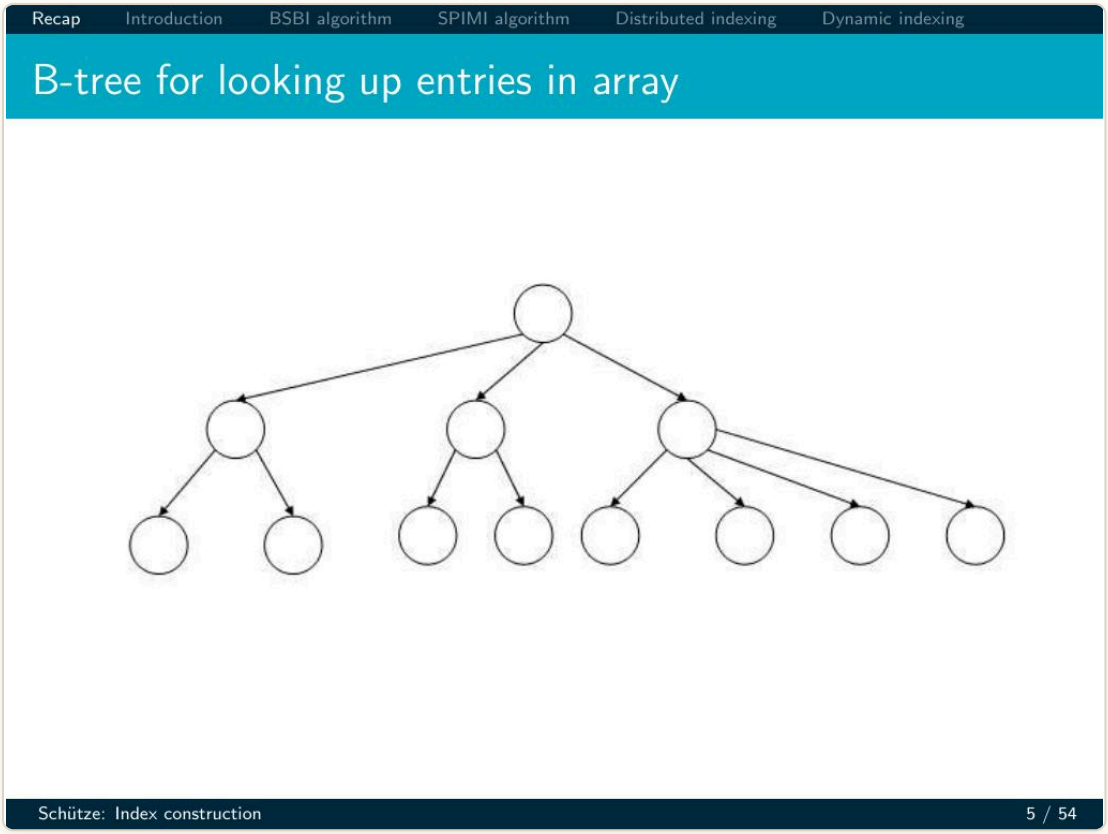


복습: 고정 폭 항목 배열로서의 사전

- term · 문서 빈도 · 포스팅 리스트 포인터 (a 656,265 / aachen 65 / … / zulu 221)
- 필요 공간: **20바이트 · 4바이트 · 4바이트**

✍ 메모

— 넘어가도 되는 쪽

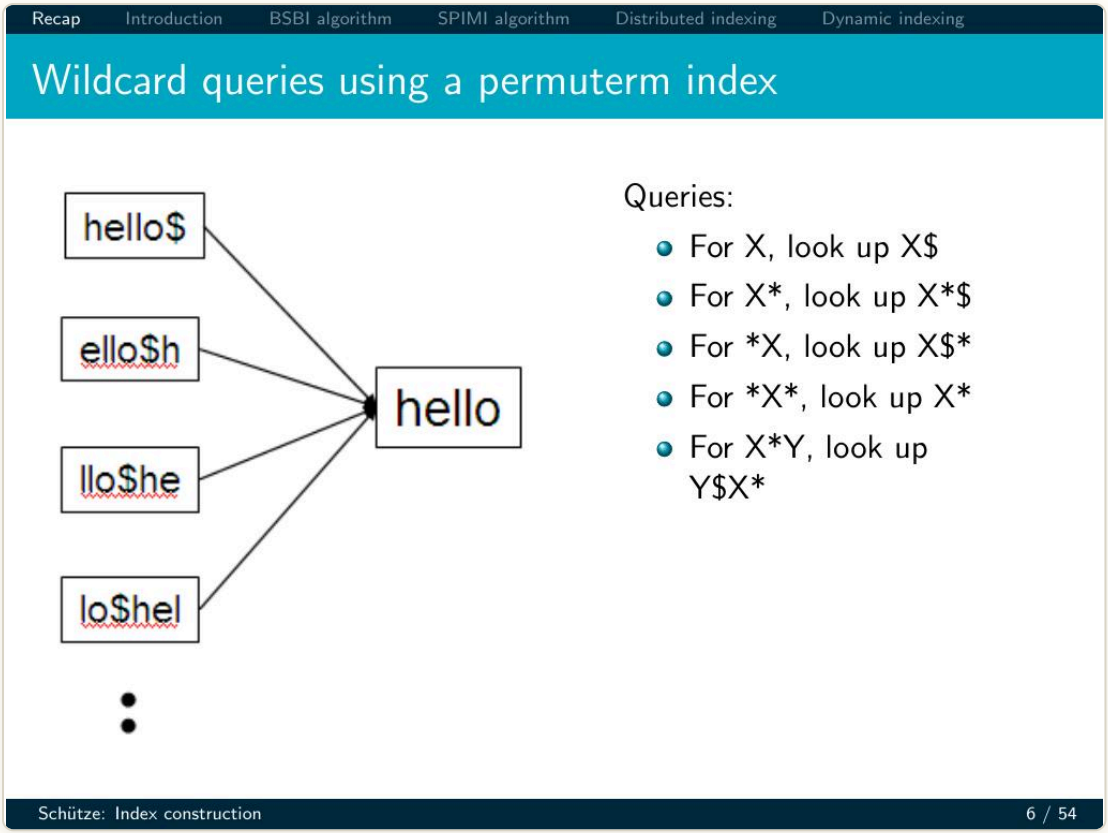


복습: 배열 항목을 찾는 B-트리

- B-트리로 사전 배열의 행을 찾는다 — 접두어 검색 가능, O(log M)

✍ 메모

— 넘어가도 되는 쪽

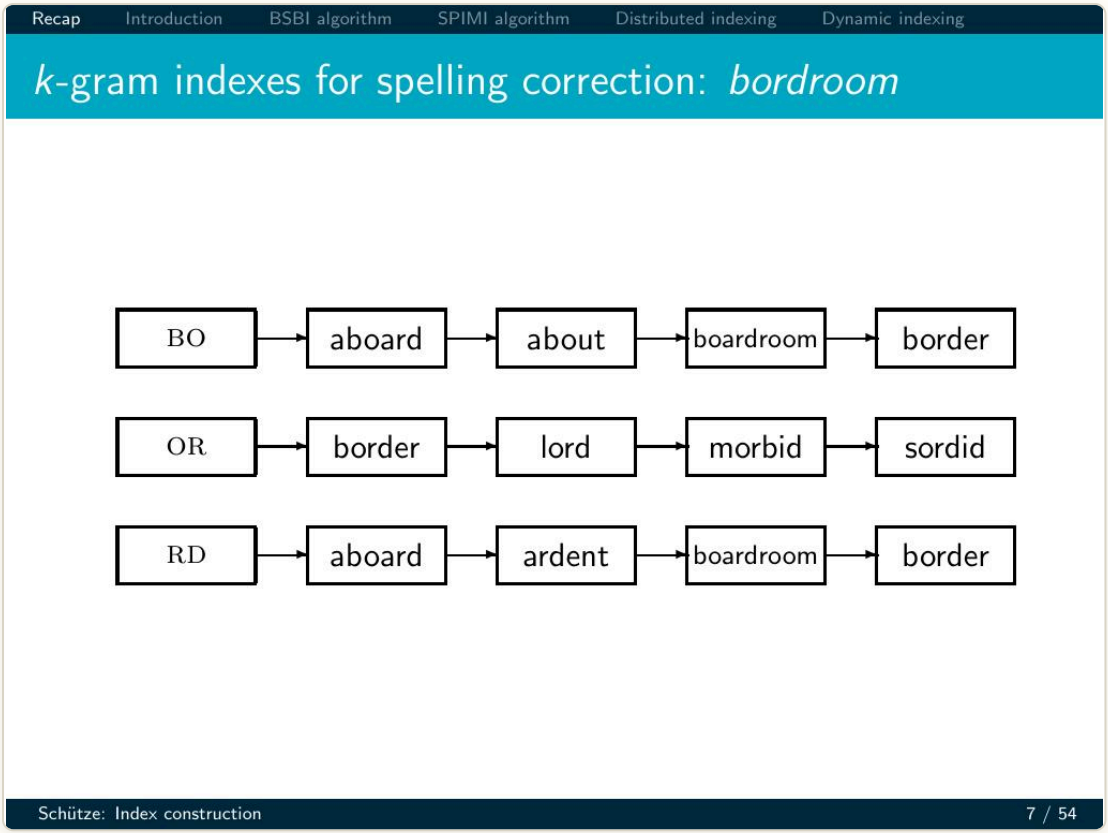


복습: permuterm 색인으로 와일드카드 질의

- X → X$
- X* → **X*$** (슬라이드 표기)
- *X → X$*
- *X* → X*
- X*Y → Y$X*

✍️ 메모

⚠ 주의 3강 p.26은 X* → $X*라고 했다. 회전 규칙대로면 $X*가 맞다 (X*$를 *가 끝에 오게 돌리면 $X*). 시험엔 $X*로 쓰자



복습: 철자 교정용 k-gram 색인 — bordroom

- BO → aboard, about, boardroom, border / OR → border, lord, morbid, sordid / RD → aboard, ardent, boardroom, border

✍ 메모

— 넘어가도 되는 쪽

Levenshtein distance for spelling correction

LEVENSHTEINDISTANCE(s_1, s_2)

```
 1  for i ← 0 to |s1|
 2  do m[i, 0] = i
 3  for j ← 0 to |s2|
 4  do m[0, j] = j
 5  for i ← 1 to |s1|
 6  do for j ← 1 to |s2|
 7      do if s1[i] = s2[j]
 8           then m[i, j] = min{m[i − 1, j] + 1, m[i, j − 1] + 1, m[i − 1, j − 1]}
 9           else m[i, j] = min{m[i − 1, j] + 1, m[i, j − 1] + 1, m[i − 1, j − 1] + 1}
10  return m[|s1|, |s2|]
```

Operations: insert, delete, replace, copy

복습: 철자 교정용 레벤슈타인 거리

- DP 표: m[i,0] = i, m[0,j] = j, 같으면 대각선+0, 다르면 대각선+1, 위·왼쪽 +1 중 최소
- 연산: 삽입, 삭제, 교체, 복사

✍ 메모

— 넘어가도 되는 쪽

Exercise: Understand Peter Norvig's spelling corrector

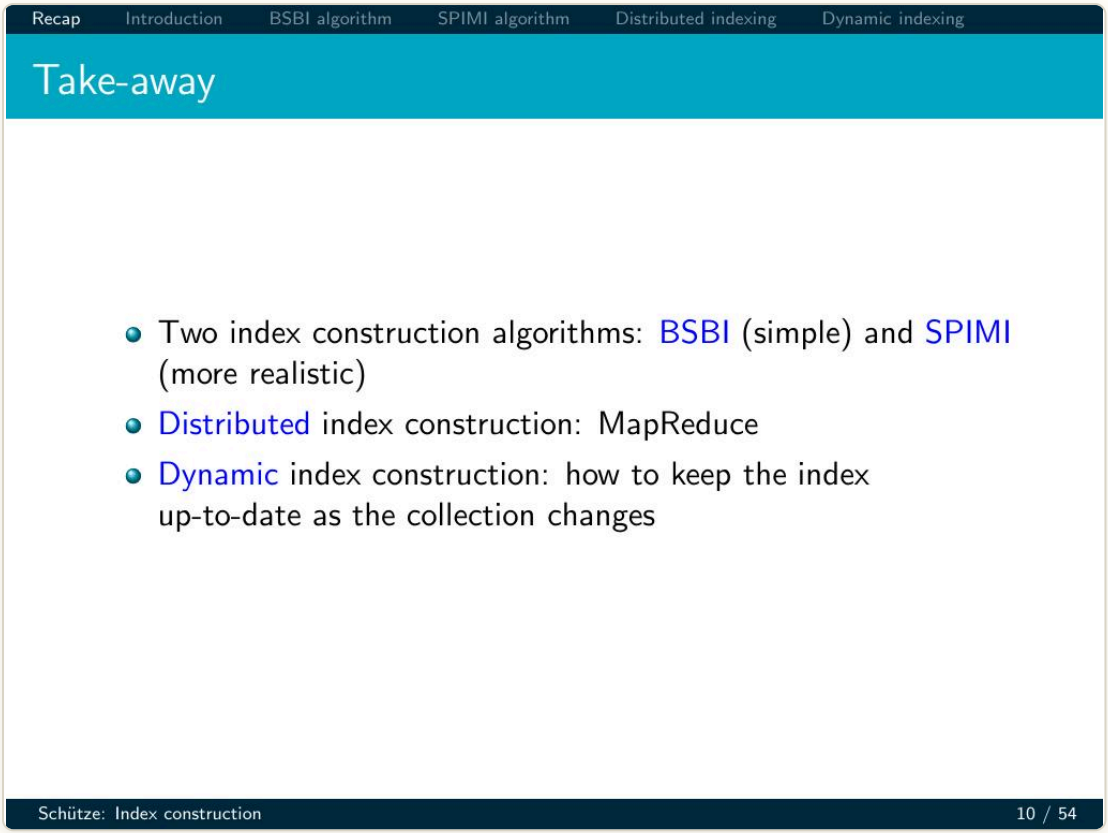
```
import re, collections
def words(text): return re.findall('[a-z]+', text.lower())
def train(features):
    model = collections.defaultdict(lambda: 1)
    for f in features:
        model[f] += 1
    return model
NWORDS = train(words(file('big.txt').read()))
alphabet = 'abcdefghijklmnopqrstuvwxyz'
def edits1(word):
   splits     = [(word[:i], word[i:]) for i in range(len(word) + 1)]
   deletes    = [a + b[1:] for a, b in splits if b]
   transposes = [a + b[1] + b[0] + b[2:] for a, b in splits if len(b) gt 1]
   replaces   = [a + c + b[1:] for a, b in splits for c in alphabet if b]
   inserts    = [a + c + b     for a, b in splits for c in alphabet]
   return set(deletes + transposes + replaces + inserts)
def known_edits2(word):
    return set(e2 for e1 in edits1(word) for e2 in
    edits1(e1) if e2 in NWORDS)
def known(words): return set(w for w in words if w in NWORDS)
def correct(word):
    candidates = known([word]) or known(edits1(word)) or
    known_edits2(word) or [word]
    return max(candidates, key=NWORDS.get)
```


복습: Peter Norvig의 철자 교정기

- edits1(거리 1 후보) → known_edits2(거리 2 중 사전에 있는 것) → 빈도가 가장 높은 후보 선택

✍ 메모

— 넘어가도 되는 쪽

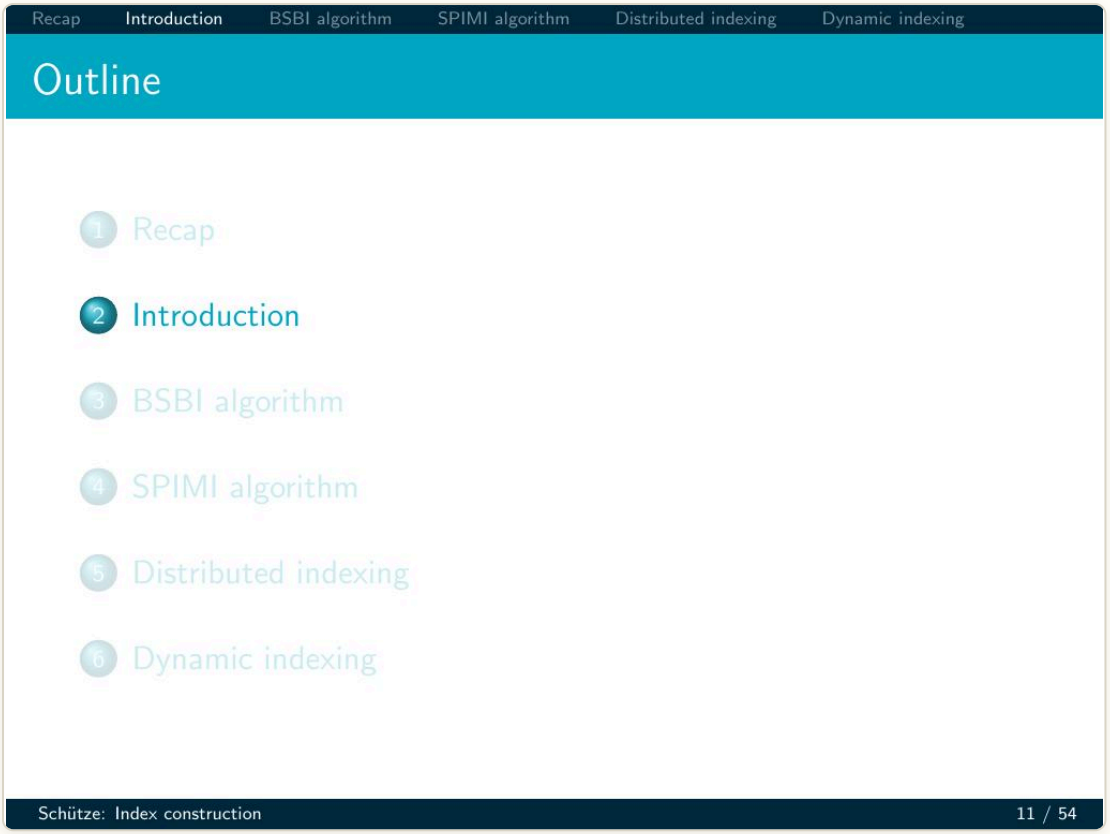


이번 강의에서 가져갈 것

- 색인 구축 알고리즘 두 가지: **BSBI**(단순)와 **SPIMI**(더 현실적)
- **분산** 색인 구축: **MapReduce**
- **동적** 색인 구축: 컬렉션이 바뀔 때 색인을 최신으로 유지하는 법

✍️ 메모

— 넘어가도 되는 쪽

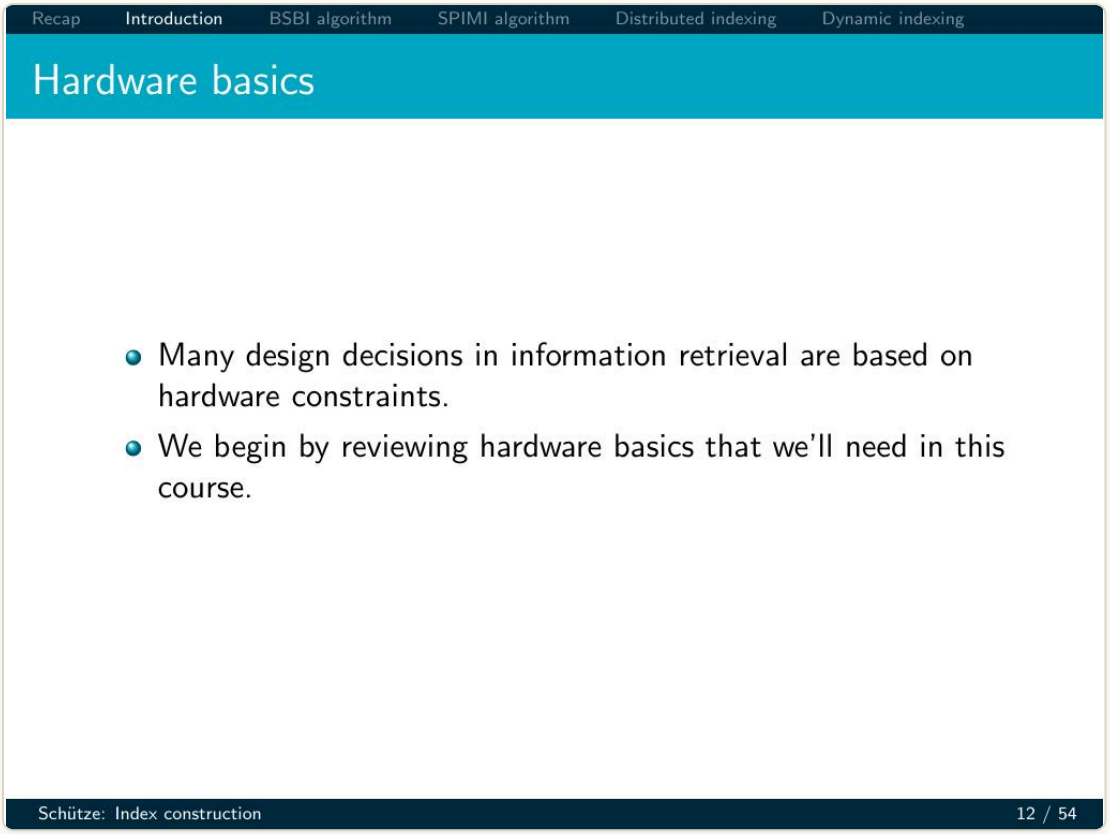


목차

- 복습 → 소개(하드웨어) → BSBI → SPIMI → 분산 색인 → 동적 색인
- 지금: **소개**

✍️ 메모

— 넘어가도 되는 쪽

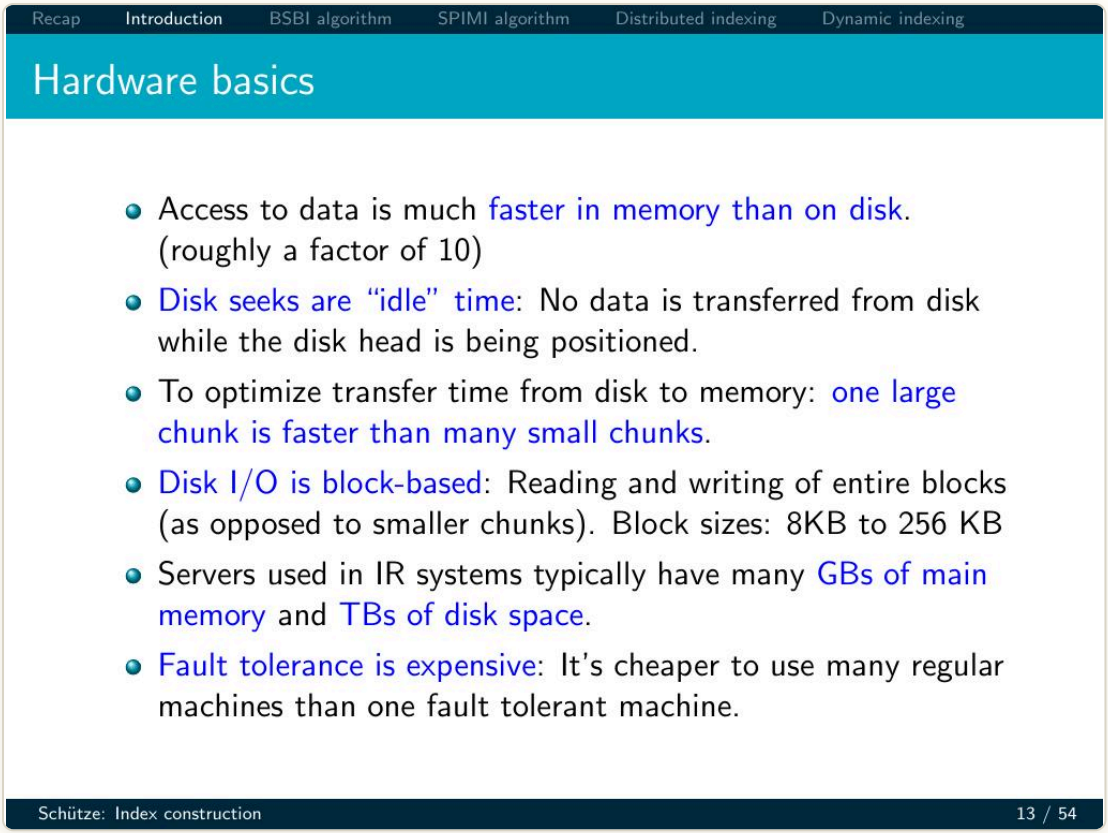


하드웨어 기초

- IR의 많은 설계 결정은 **하드웨어 제약**에 기반한다
- 이 수업에 필요한 하드웨어 기초부터 복습

✍ 메모

— 넘어가도 되는 쪽

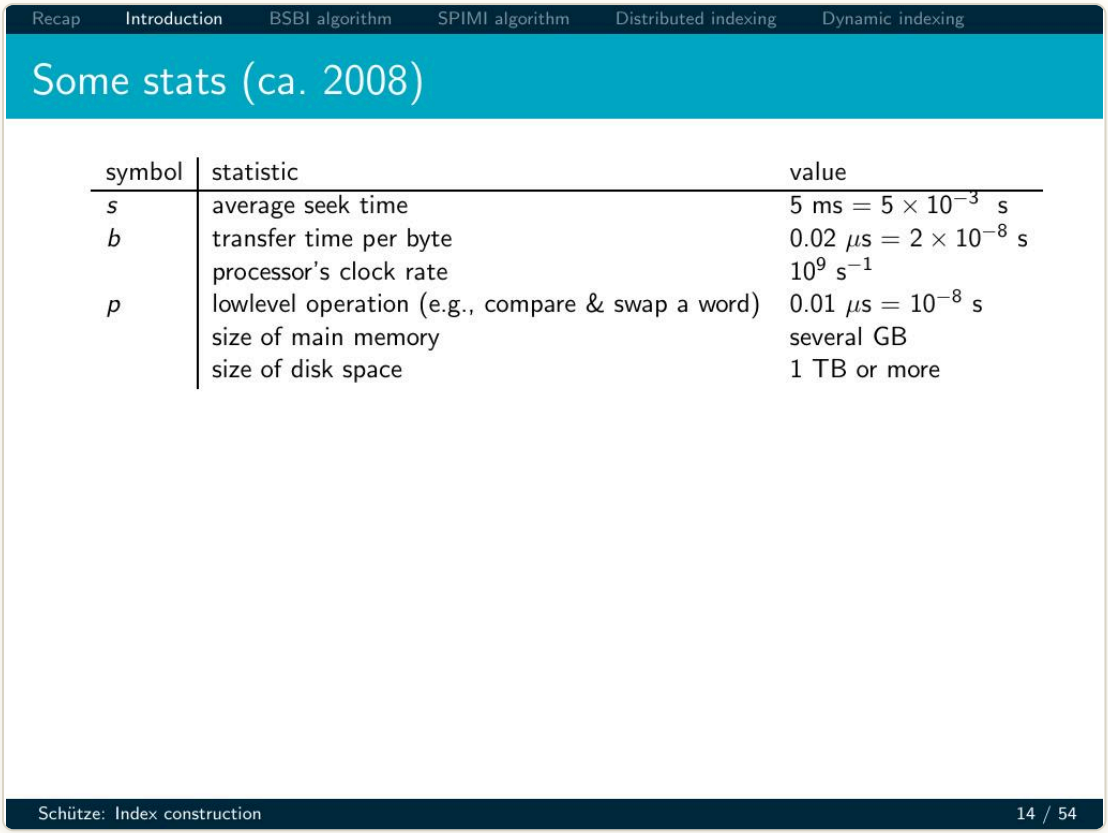


하드웨어 기초 (계속)

- **메모리**의 데이터 접근이 **디스크**보다 훨씬 빠르다 (대략 10배)
- **디스크 탐색(seek)**은 '노는' 시간: 헤드를 옮기는 동안엔 데이터가 전송되지 않는다
- 디스크→메모리 전송 최적화: **큰 덩어리 하나**가 작은 덩어리 여러 개보다 빠르다
- 디스크 I/O는 **블록 단위**: 블록 전체를 읽고 쓴다. 블록 크기 **8KB ~ 256KB**
- IR 서버는 보통 수 GB 메인 메모리, 수 TB 디스크
- **결함 허용은 비싸다**: 결함 허용 기계 한 대보다 **보통 기계 여러 대**가 싸다

✍ 메모

★ 시험 디스크: seek 최소화 + 큰 블록 연속 읽기 → BSBI·SPIMI 설계의 이유

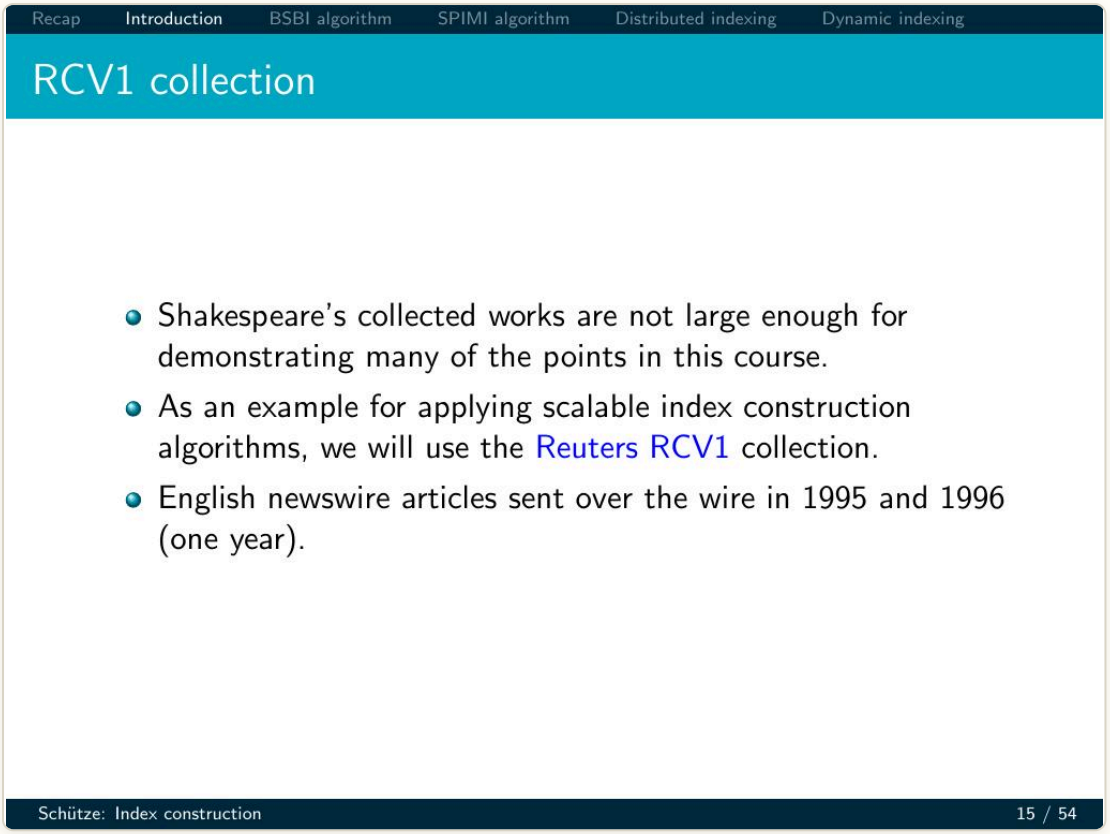


몇 가지 수치 (2008년경)

- s 평균 탐색 시간: **5ms** = 5 × 10^{-3} s
- b 바이트당 전송 시간: **0.02μs** = 2 × 10^{-8} s
- 프로세서 클록: 10^{9} s^{-1}
- p 저수준 연산(예: 단어 비교·교환): 0.01μs = 10^{-8} s
- 메인 메모리: 수 GB / 디스크: 1TB 이상

✍️ 메모

✏ 예·풀이 탐색 1번(5ms) 동안 약 250KB를 전송할 수 있다 → 탐색이 매우 비싸다

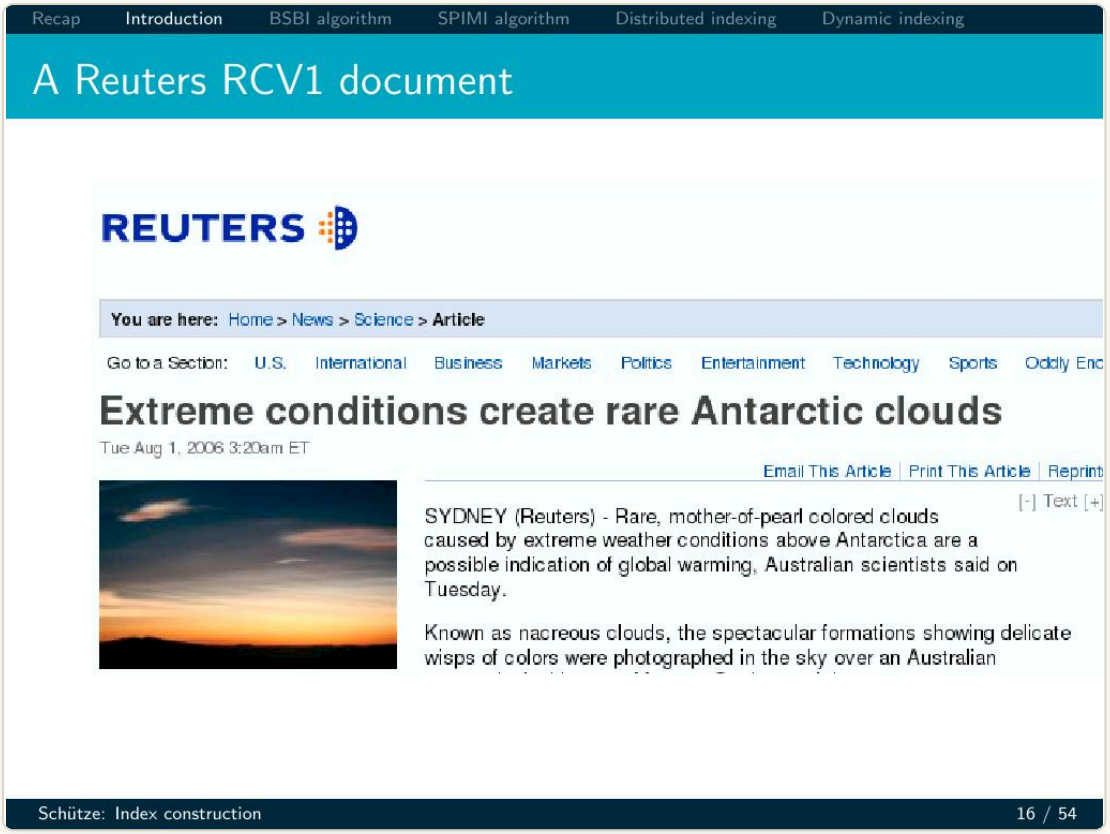


RCV1 컬렉션

- 셰익스피어 전집은 이 수업의 많은 요점을 보이기엔 **너무 작다**
- 확장 가능한 색인 구축 알고리즘의 예로 **로이터 RCV1** 컬렉션을 쓴다
- 1995~1996년(1년간) 통신으로 보낸 **영어 뉴스 기사**

✍ 메모

— 넘어가도 되는 쪽

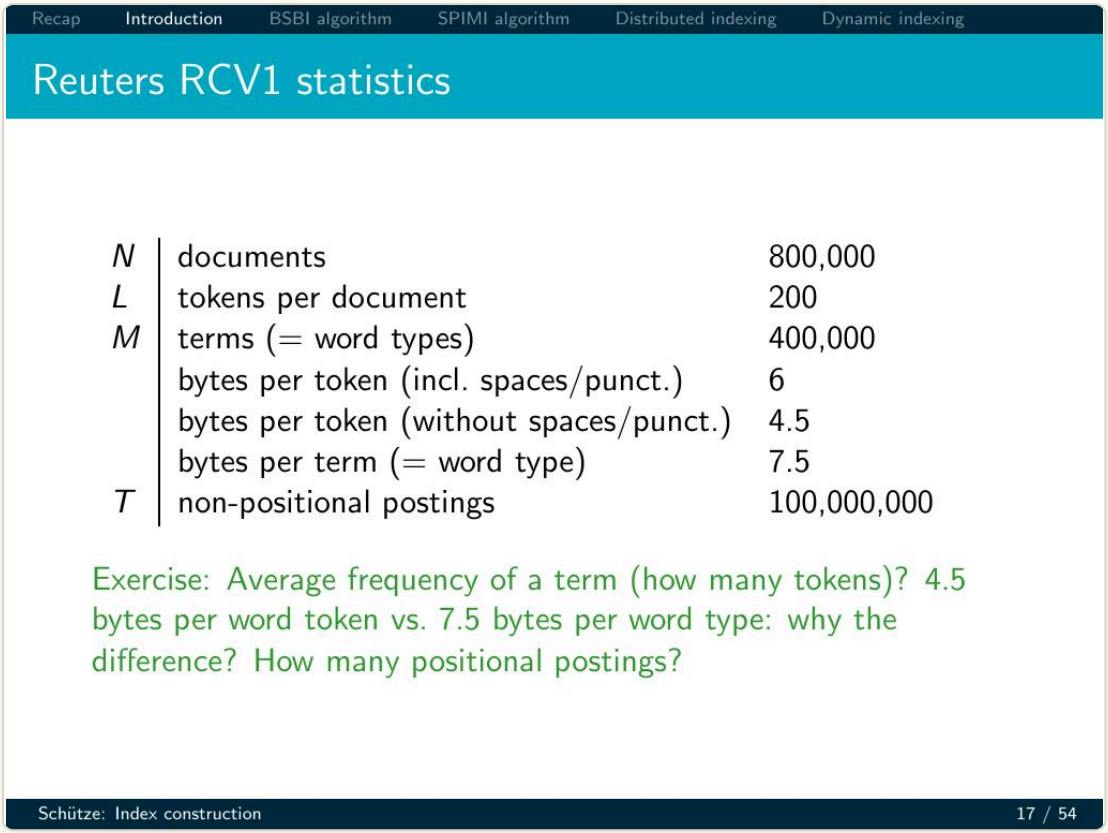


로이터 RCV1 문서 예

- 기사 화면: 'Extreme conditions create rare Antarctic clouds' (2006-08-01) — 남극 위 진주모 구름

✍ 메모

— 넘어가도 되는 쪽



로이터 RCV1 통계

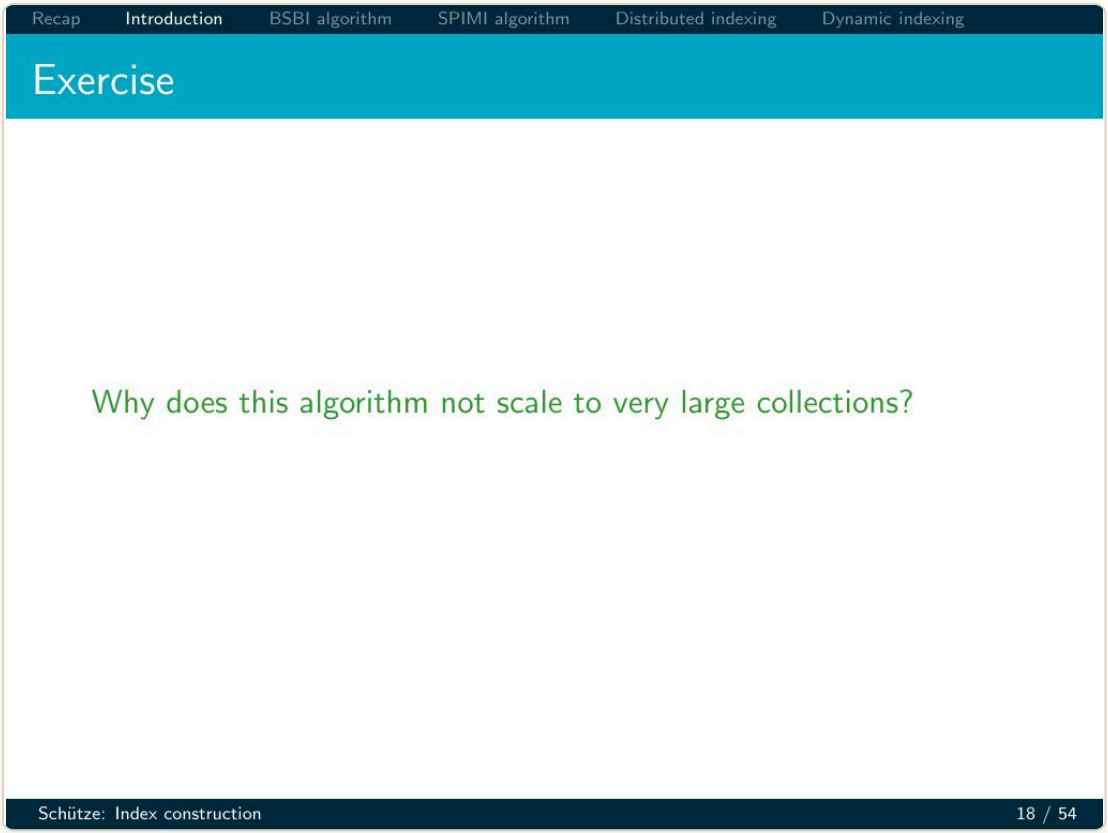
- **N** 문서 **800,000**
- **L** 문서당 토큰 **200**
- **M** 단어(= 단어 유형) **400,000**
- 토큰당 바이트(공백·문장부호 포함) 6
- 토큰당 바이트(제외) **4.5**
- 단어(유형)당 바이트 **7.5**
- **T** 비위치 포스팅 **100,000,000**
- 연습: 단어의 평균 빈도(토큰 몇 개)? 토큰당 4.5바이트 vs 유형당 7.5바이트 — 왜 다를까? 위치 포스팅은 몇 개?

✍ 메모

✏ 예·풀이 평균 빈도 = N·L / M = 800,000 × 200 / 400,000 = 400 토큰

✏ 예·풀이 4.5 < 7.5: 자주 나오는 단어(the, a …)가 짧아서 토큰 평균은 짧다

✏ 예·풀이 위치 포스팅 = 토큰 수 = 800,000 × 200 = 1억 6천만

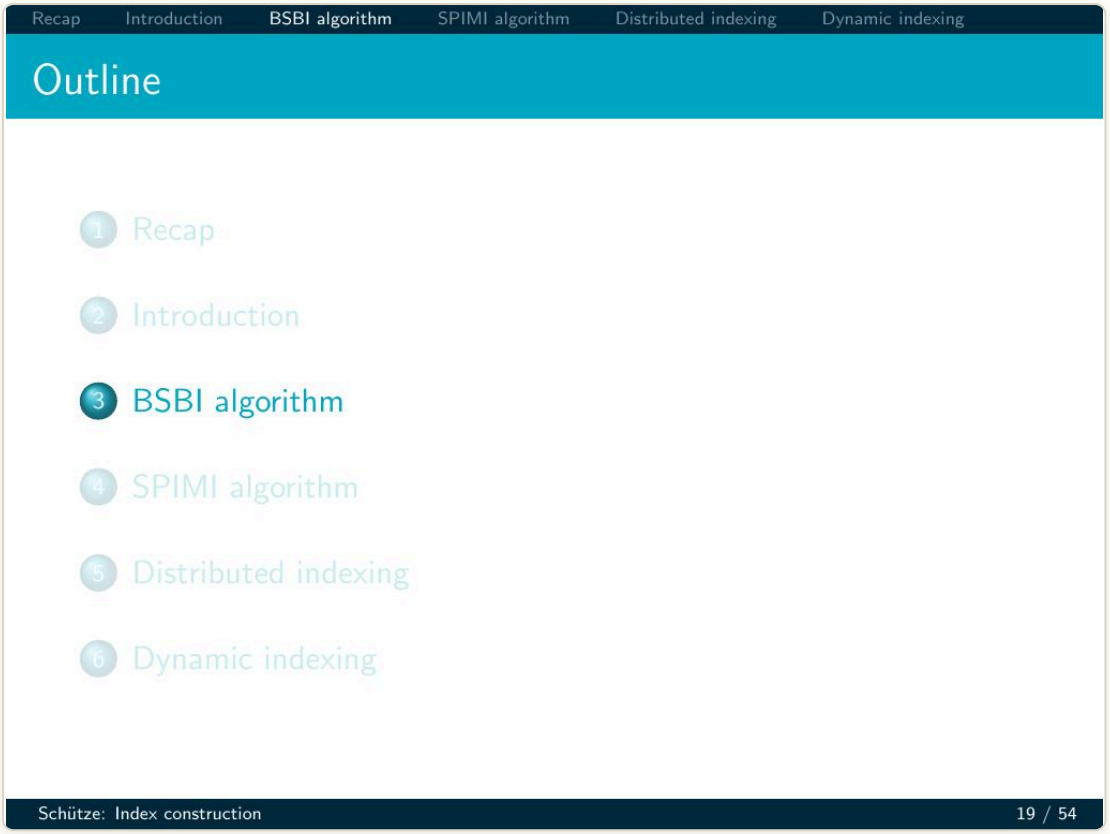


연습

- (1강의 메모리 정렬 방식) 이 알고리즘은 왜 아주 큰 컬렉션으로 **확장되지 않을까?**

✍️ 메모

💡 이해 답: 모든 포스팅을 메모리에 담아 정렬해야 하는데 메모리가 모자란다

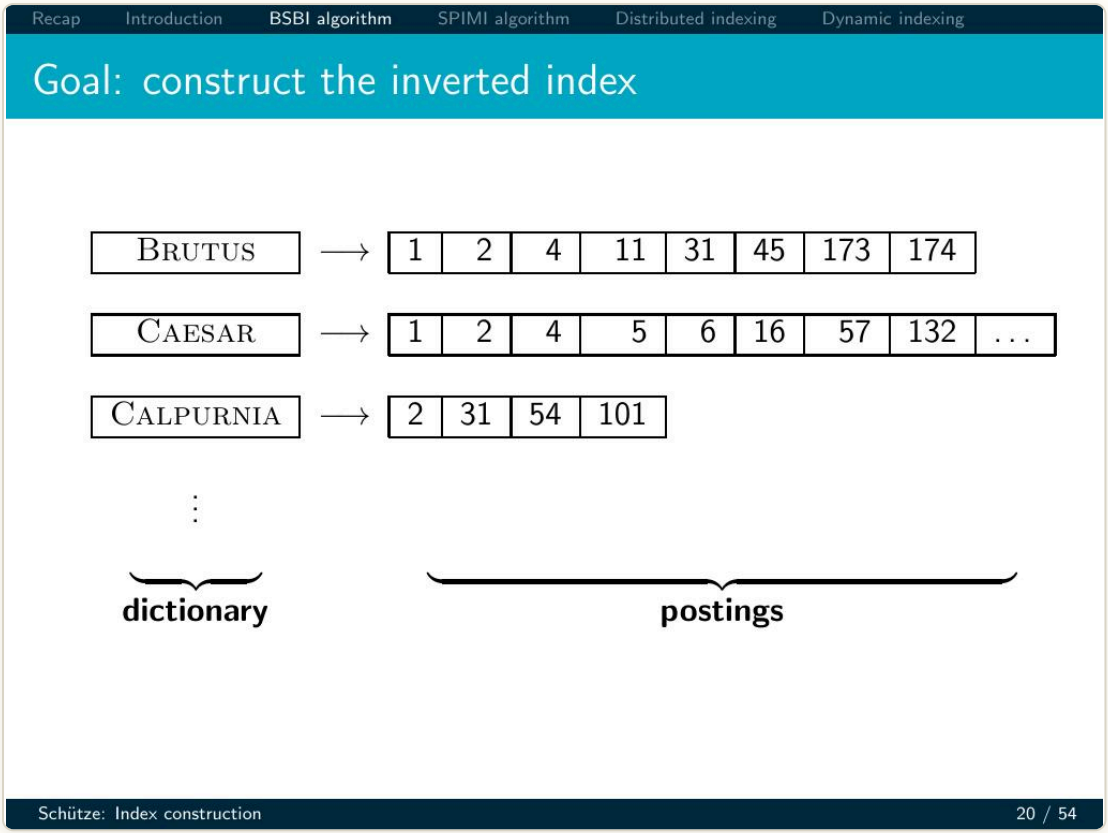


목차

- 복습 → 소개(하드웨어) → BSBI → SPIMI → 분산 색인 → 동적 색인
- 지금: **BSBI 알고리즘**

✍️ 메모

— 넘어가도 되는 쪽

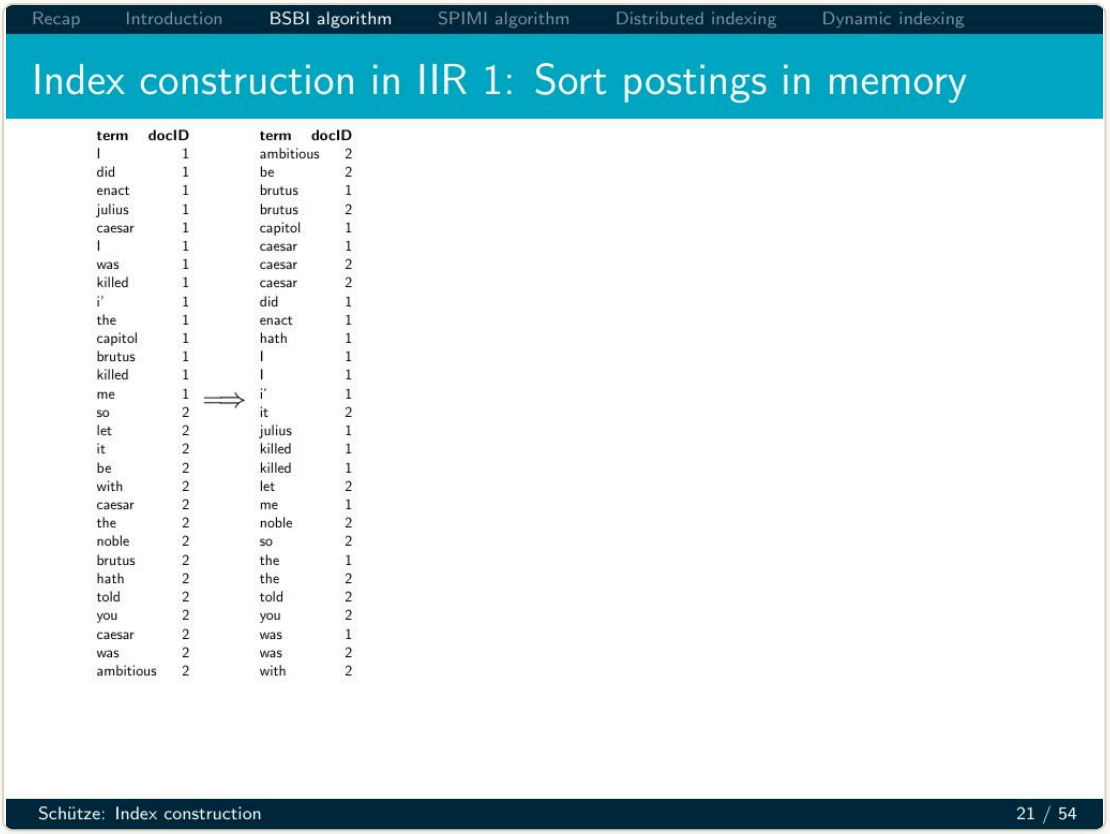


목표: 역색인 구축

- Brutus → 1 2 4 11 31 45 173 174 / Caesar → … / Calpurnia → 2 31 54 101 — 사전 + 포스팅

✍ 메모

— 넘어가도 되는 쪽

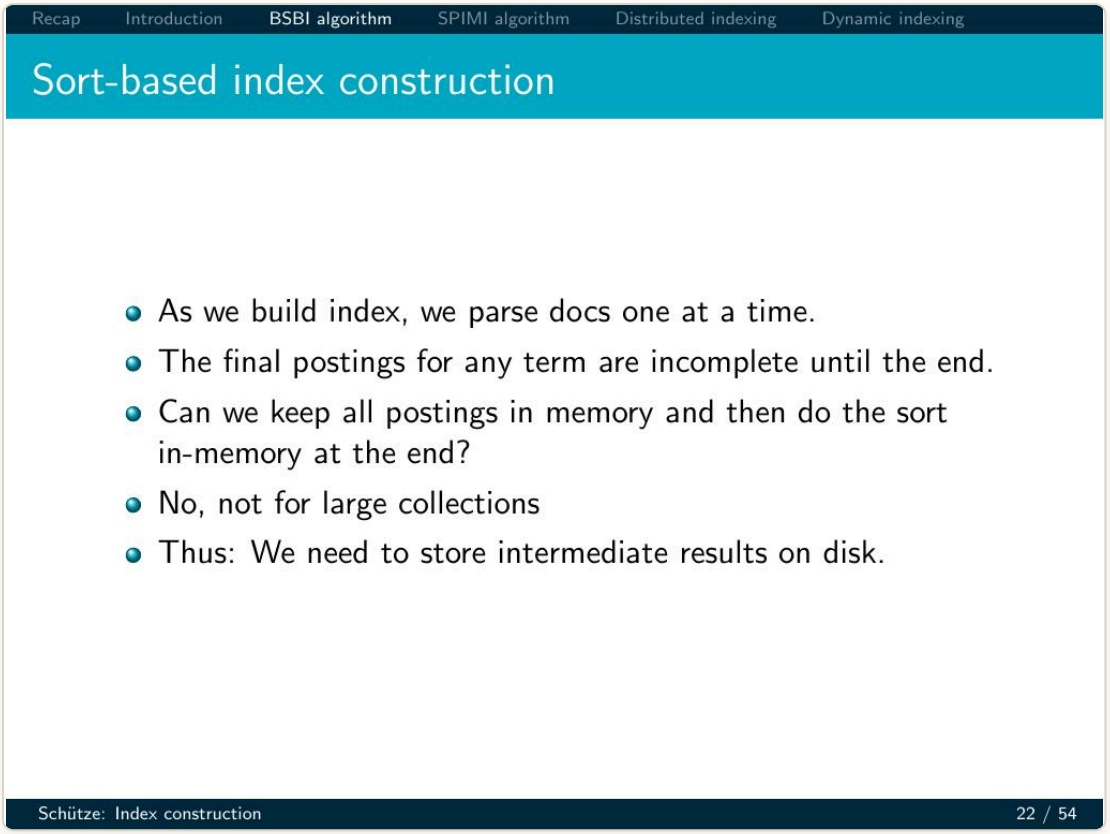


1강의 색인 구축: 메모리에서 포스팅 정렬

- (term, docID) 쌍을 만들어 **메모리에서** term·docID순으로 정렬

✍ 메모

— 넘어가도 되는 쪽

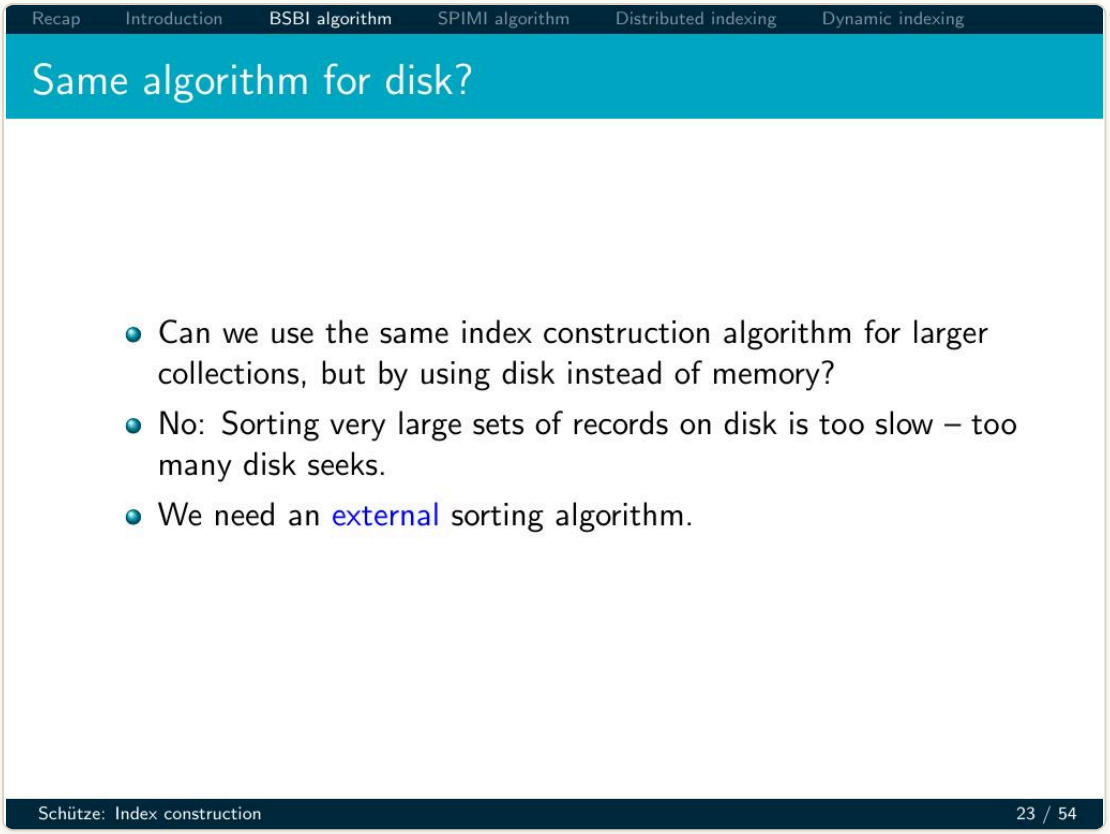


정렬 기반 색인 구축

- 색인을 만들며 문서를 **하나씩** 파싱
- 어떤 단어의 최종 포스팅은 **끝날 때까지 미완성**
- 모든 포스팅을 메모리에 두고 마지막에 메모리에서 정렬할 수 있을까?
  - **아니다**, 큰 컬렉션에선 안 된다
- 그래서 **중간 결과를 디스크에** 저장해야

✍️ 메모

— 넘어가도 되는 쪽

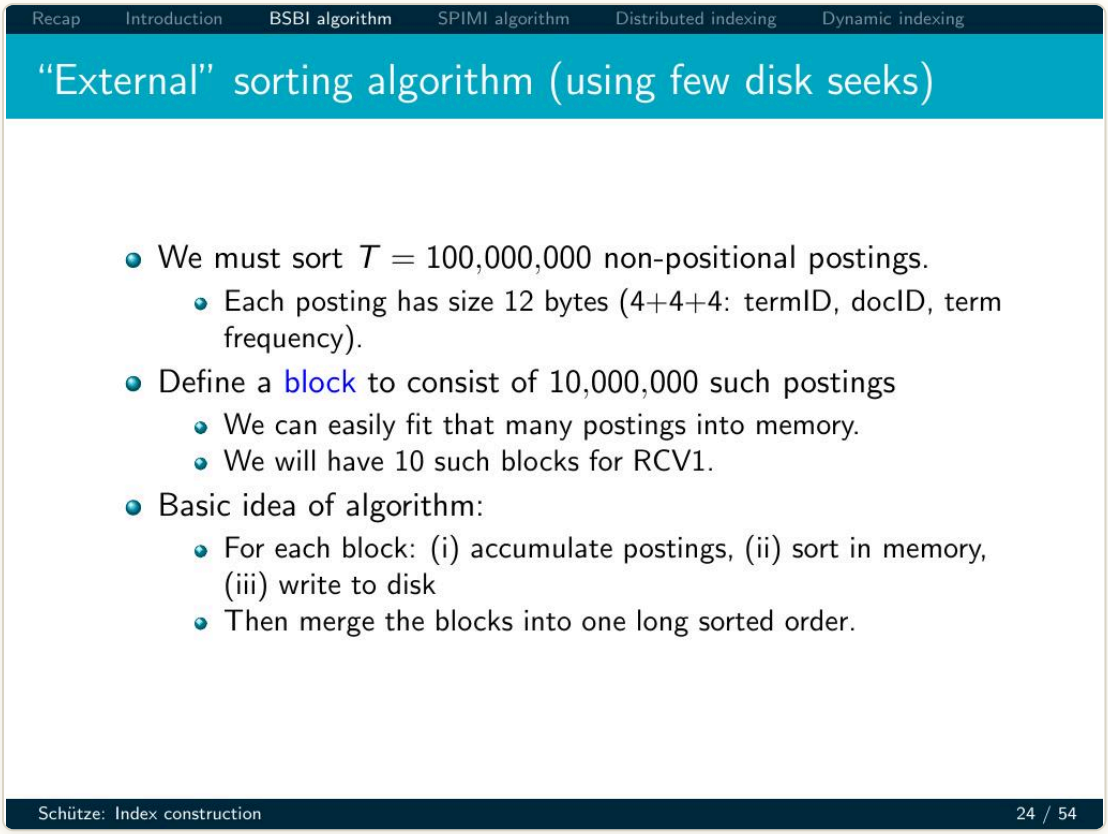


디스크에서 같은 알고리즘?

- 메모리 대신 디스크로 같은 알고리즘을 쓸 수 있을까?
- **아니다**: 디스크에서 아주 큰 레코드 집합을 정렬하면 **너무 느리다** — 디스크 탐색이 너무 많다
- **외부 정렬(external sorting)** 알고리즘이 필요

✍ 메모

— 넘어가도 되는 쪽



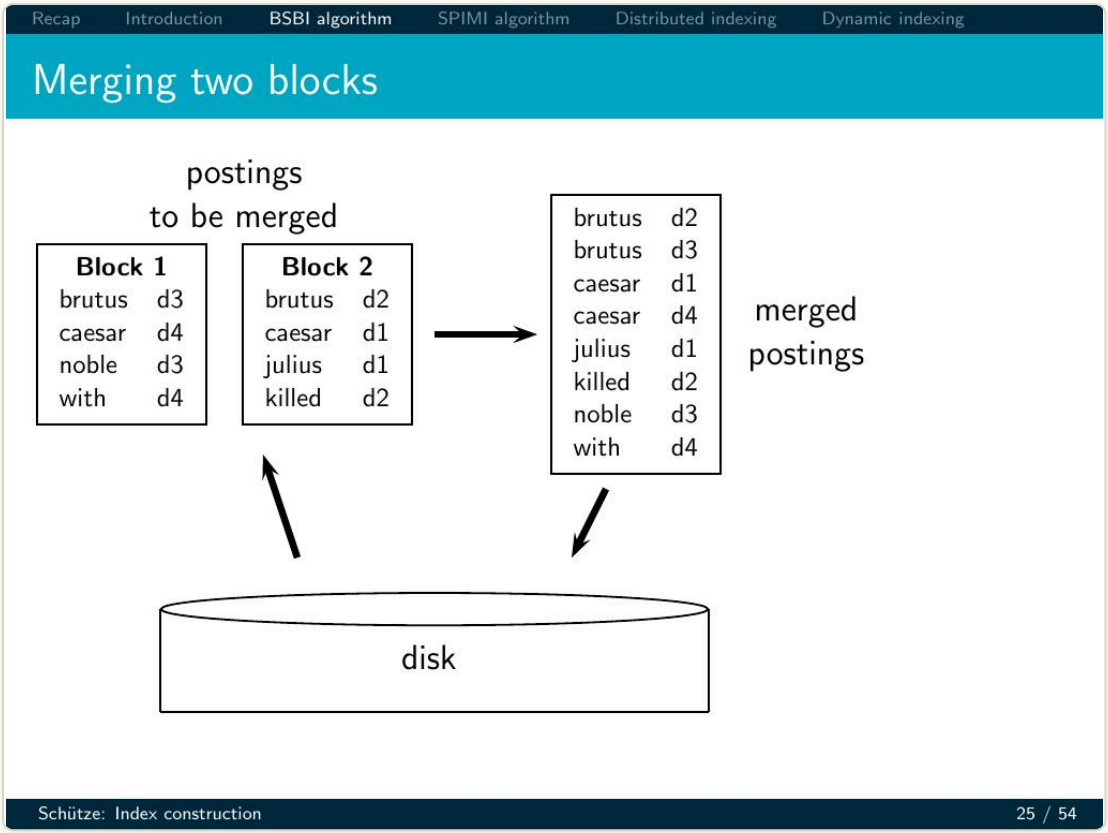
'외부' 정렬 알고리즘 (디스크 탐색 적게)

- 비위치 포스팅 **T = 1억 개**를 정렬해야
- 포스팅 하나 **12바이트** (4 + 4 + 4: termID, docID, 단어 빈도)
- 블록 하나 = 포스팅 **천만 개** — 메모리에 쉽게 들어간다
- RCV1엔 이런 블록이 **10개**
- 기본 아이디어
  - 블록마다: (i) 포스팅 모으기 (ii) **메모리에서 정렬** (iii) 디스크에 쓰기
  - 그다음 블록들을 **병합**해 하나의 긴 정렬 순서로

✍ 메모

★ 시험 BSBI = 블록별 정렬 후 디스크 → 마지막에 병합

✏ 예·풀이 1억 / 천만 = 10블록, 포스팅 12B × 1억 = 1.2GB

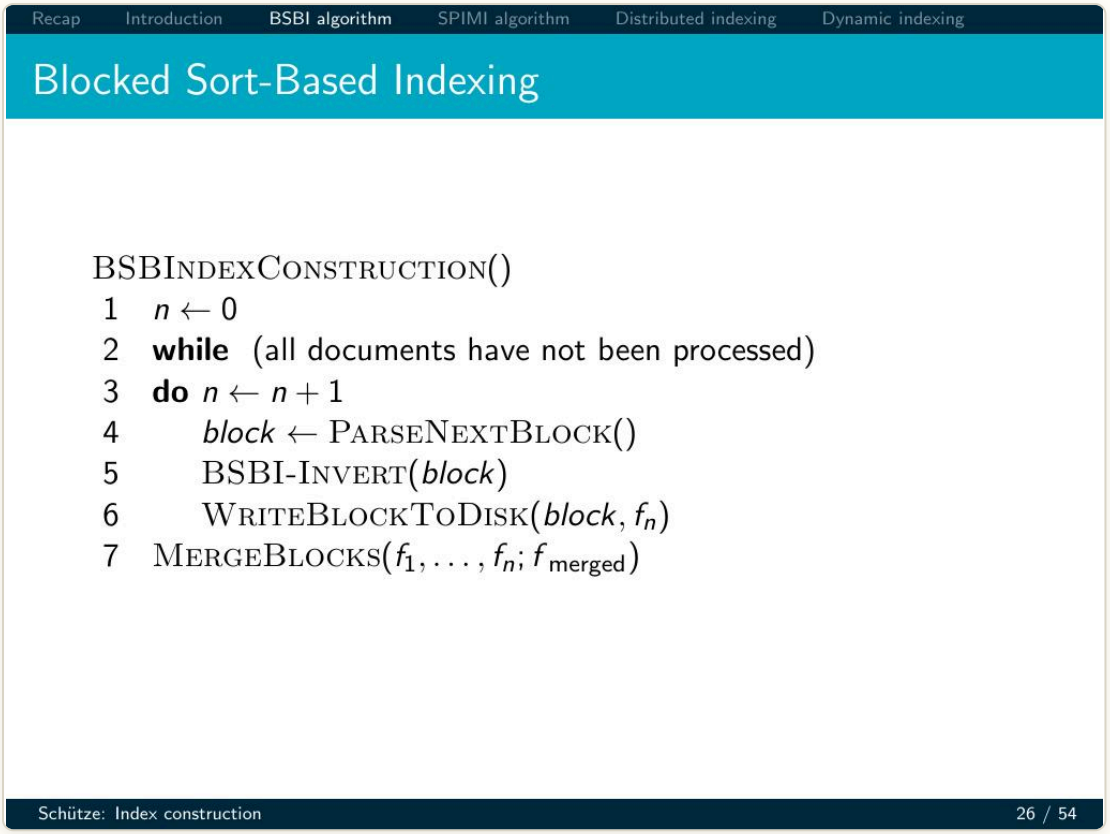


두 블록 병합 (그림)

- Block 1: brutus d3, caesar d4, noble d3, with d4
- Block 2: brutus d2, caesar d1, julius d1, killed d2
- → 병합: brutus d2, brutus d3, caesar d1, caesar d4, julius d1, killed d2, noble d3, with d4
- 디스크에서 읽어 병합한 결과를 다시 디스크에 씀

✍ 메모

— 넘어가도 되는 쪽

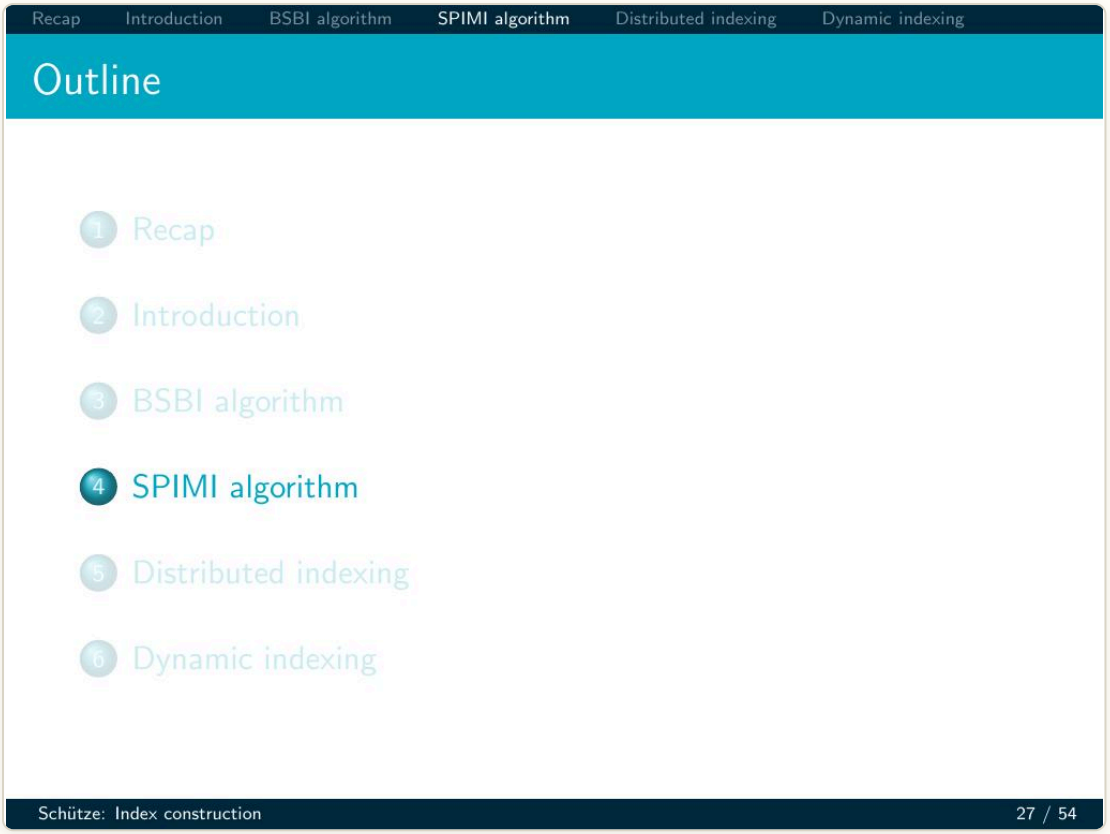


블록 정렬 기반 색인 (BSBI)

```
BSBINDEXCONSTRUCTION()
n ← 0
while (아직 처리 안 된 문서가 있으면)
  n ← n + 1
  block ← PARSENEXTBLOCK()
  BSBI-INVERT(block)
  WRITEBLOCKTODISK(block, fn)
MERGEBLOCKS(f1, …, fn; f_merged)
```

✍ 메모

★ 시험 BSBI = Blocked Sort-Based Indexing. termID를 쓰므로 사전(term→termID)을 메모리에 유지해야 함

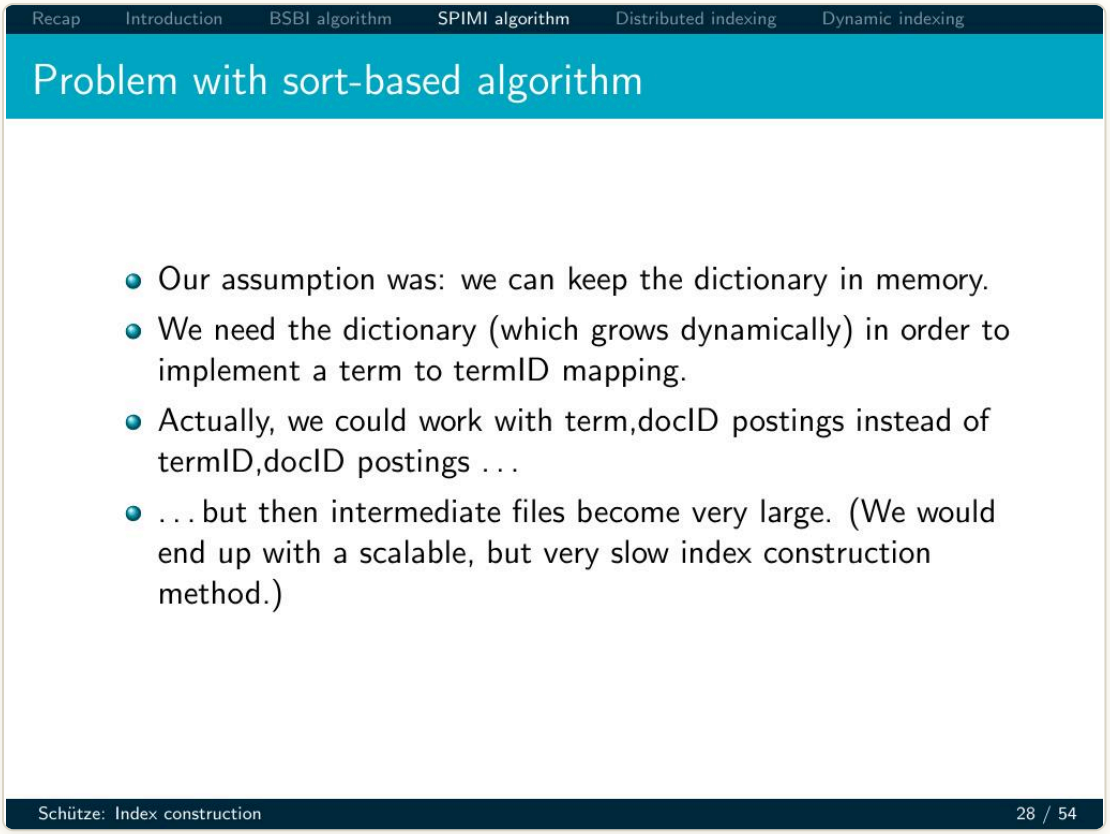


목차

- 복습 → 소개(하드웨어) → BSBI → SPIMI → 분산 색인 → 동적 색인
- 지금: **SPIMI 알고리즘**

✍️ 메모

— 넘어가도 되는 쪽

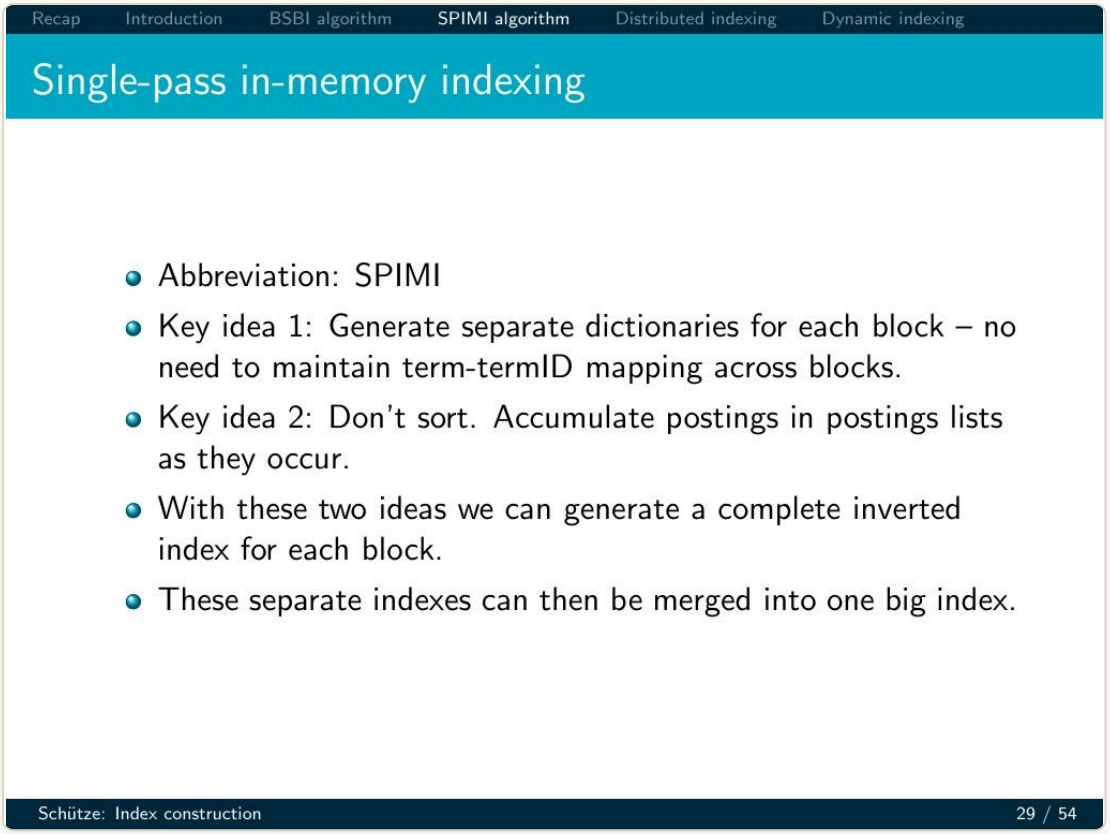


정렬 기반 알고리즘의 문제

- 가정: **사전을 메모리에** 둘 수 있다
- term → termID 대응을 하려면 (계속 커지는) 사전이 필요
- termID 대신 (term, docID) 포스팅으로 할 수도 있지만 ...
- ... 그러면 **중간 파일이 매우 커진다** (확장은 되지만 매우 느린 방법)

✍ 메모

★ 시험 **BSBI의 약점 = 전체 term→termID 사전을 메모리에**

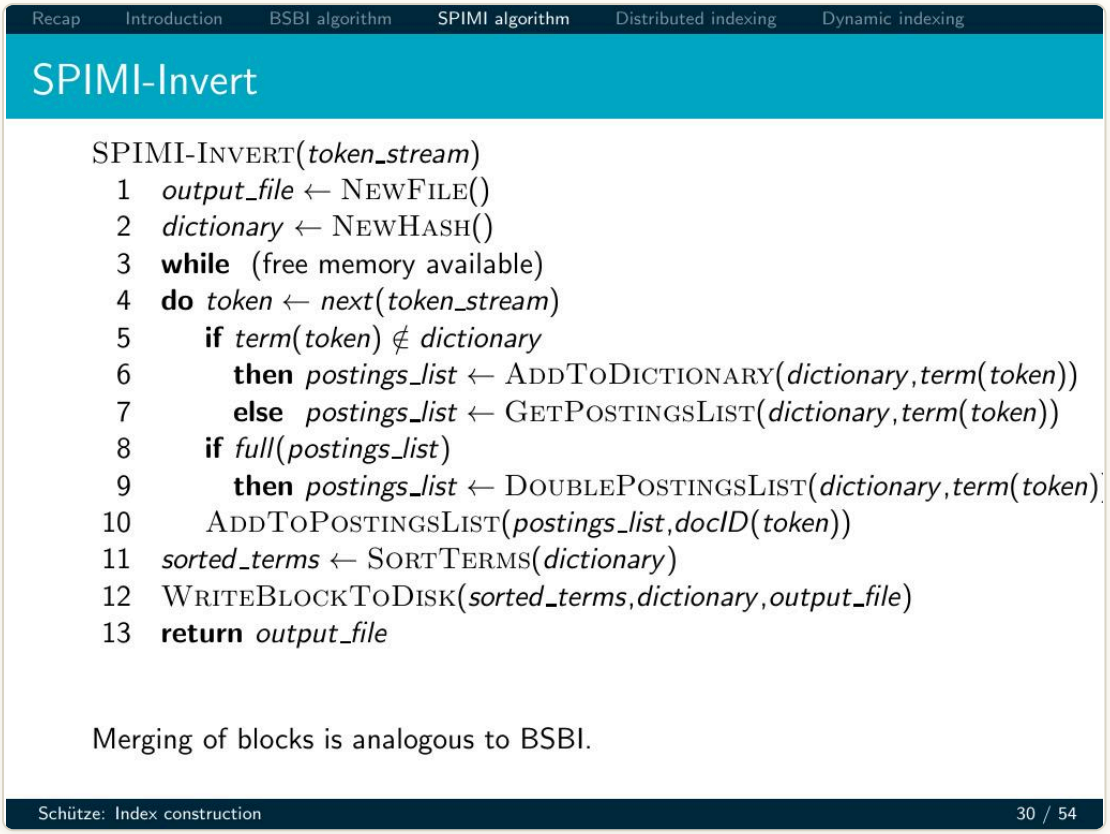


단일 패스 메모리 색인 (SPIMI)

- 약어: **SPIMI** (Single-pass in-memory indexing)
- 핵심 아이디어 1: **블록마다 별도 사전** — 블록 사이에 term-termID 대응을 유지할 필요 없음
- 핵심 아이디어 2: **정렬하지 않는다**. 포스팅이 나오는 대로 포스팅 리스트에 **쌓는다**
- 이 두 아이디어로 블록마다 **완전한 역색인**을 만든다
- 이 별도 색인들을 하나의 큰 색인으로 **병합**

✍ 메모

★ 시험 SPIMI 핵심 2개: 블록별 사전 + 포스팅 정렬 안 함 (단어만 블록 끝에 정렬)



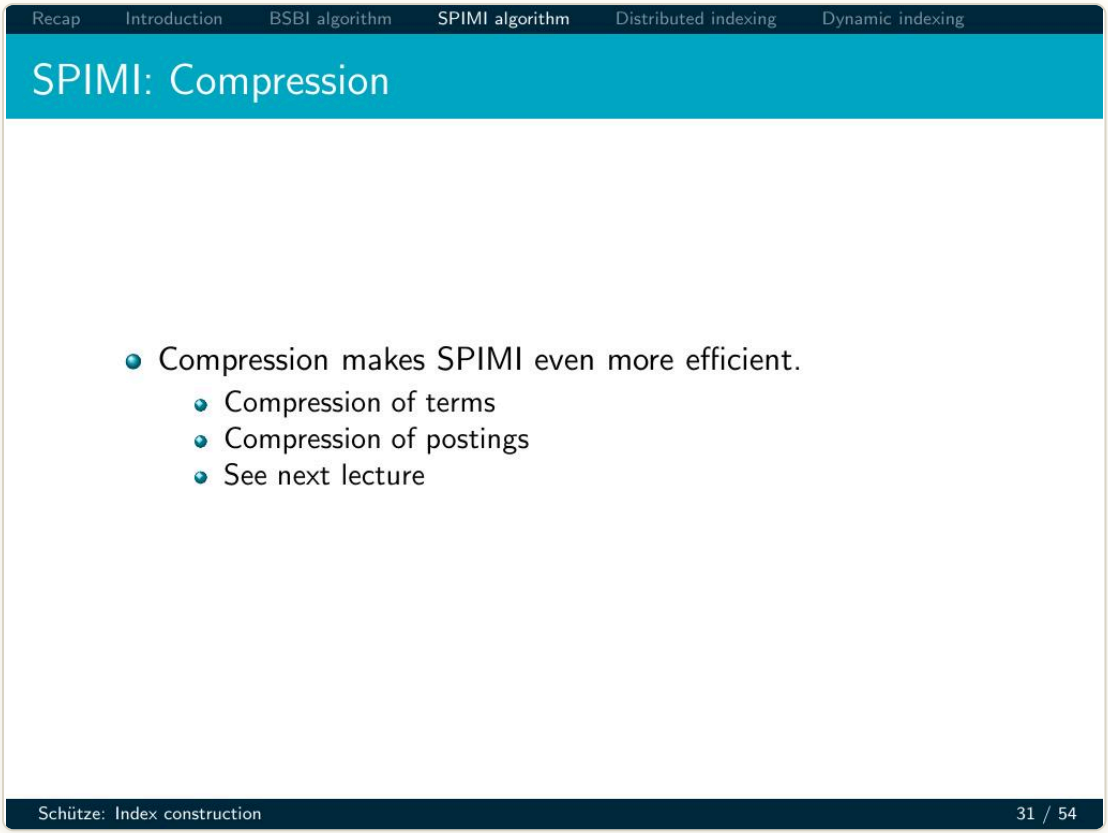
SPIMI-INVERT

```
output_file ← NEWFILE(); dictionary ← NEWHASH()
while (빈 메모리가 있으면)
  token ← next(token_stream)
  term(token)이 사전에 없으면 ADDTODICTIONARY, 있으면 GETPOSTINGSLIST
  포스팅 리스트가 꽉 차면 DOUBLEPOSTINGSLIST (두 배로)
  ADDTOPOSTINGSLIST(postings_list, docID(token))
sorted_terms ← SORTTERMS(dictionary)
WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
```

- 블록 병합은 BSBI와 같다

✍ 메모

💡 이해 사전은 해시, 블록을 쓰기 직전에 '단어만' 정렬 (병합하기 쉽게)

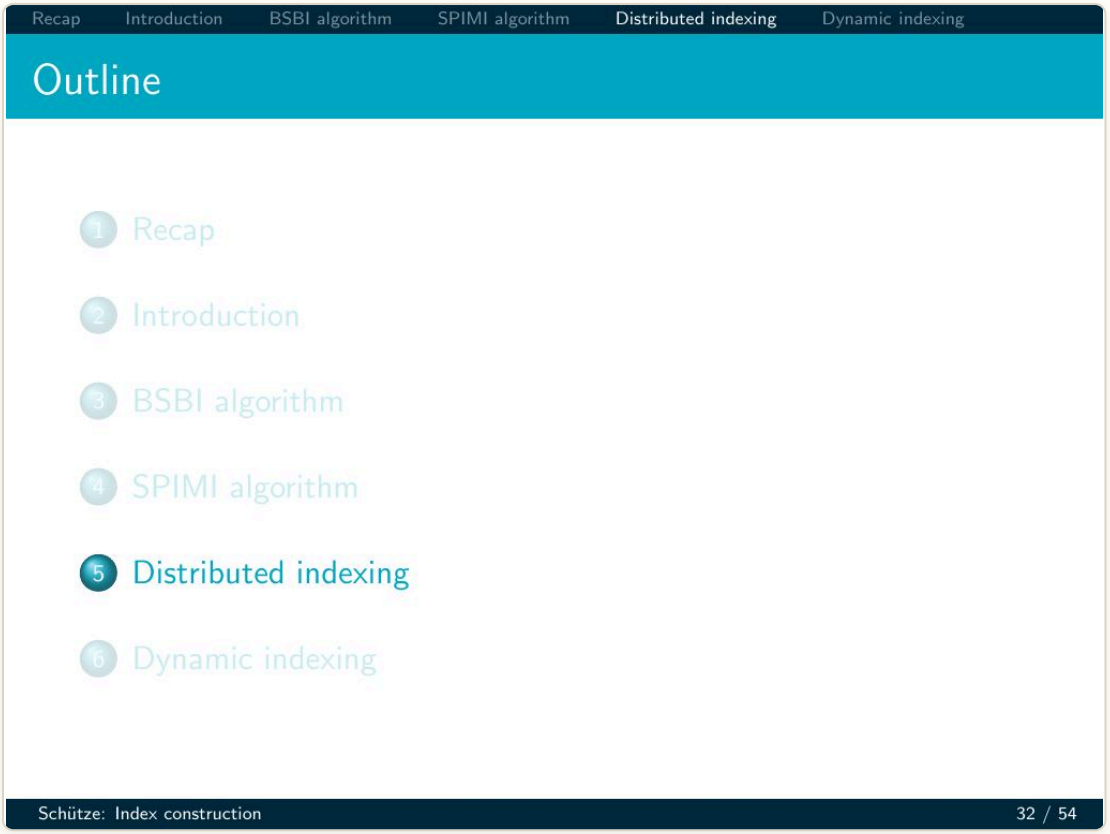


SPIMI: 압축

- **압축**하면 SPIMI가 더 효율적
  - 단어 압축
  - 포스팅 압축
- 다음 강의(5강) 참고

✍ 메모

— 넘어가도 되는 쪽

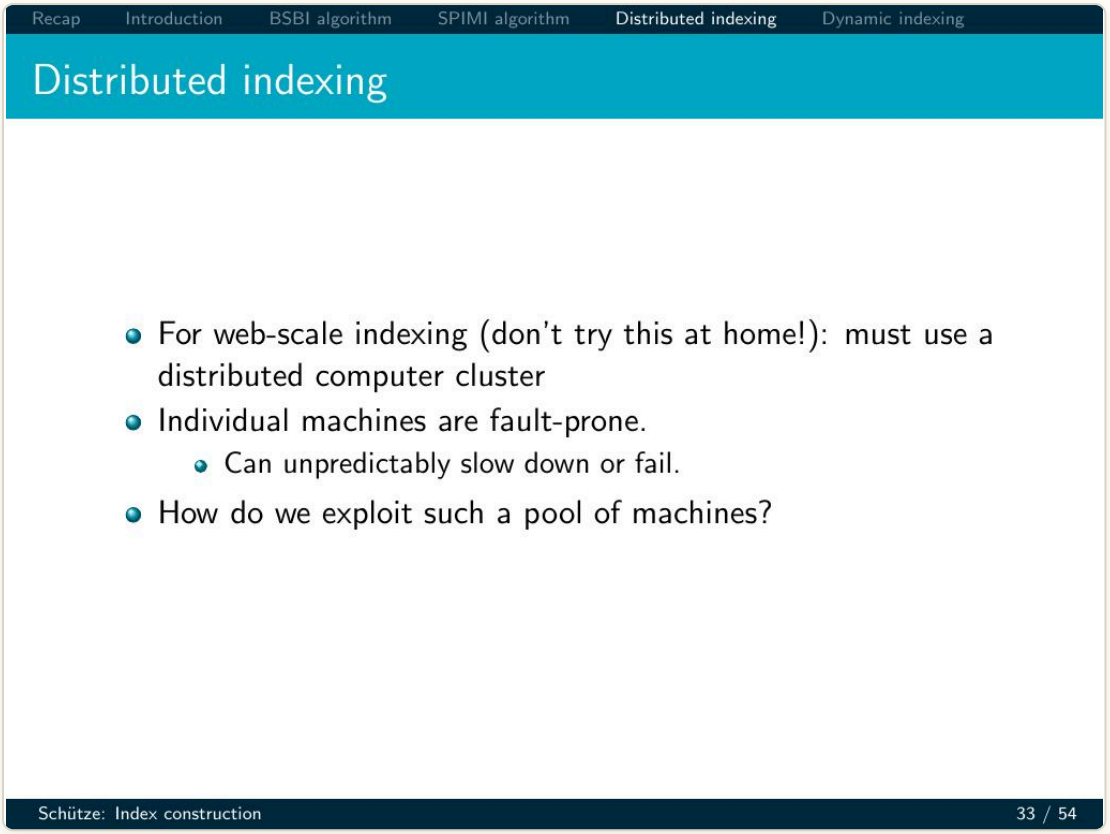


목차

- 복습 → 소개(하드웨어) → BSBI → SPIMI → 분산 색인 → 동적 색인
- 지금: **분산 색인**

✍ 메모

— 넘어가도 되는 쪽

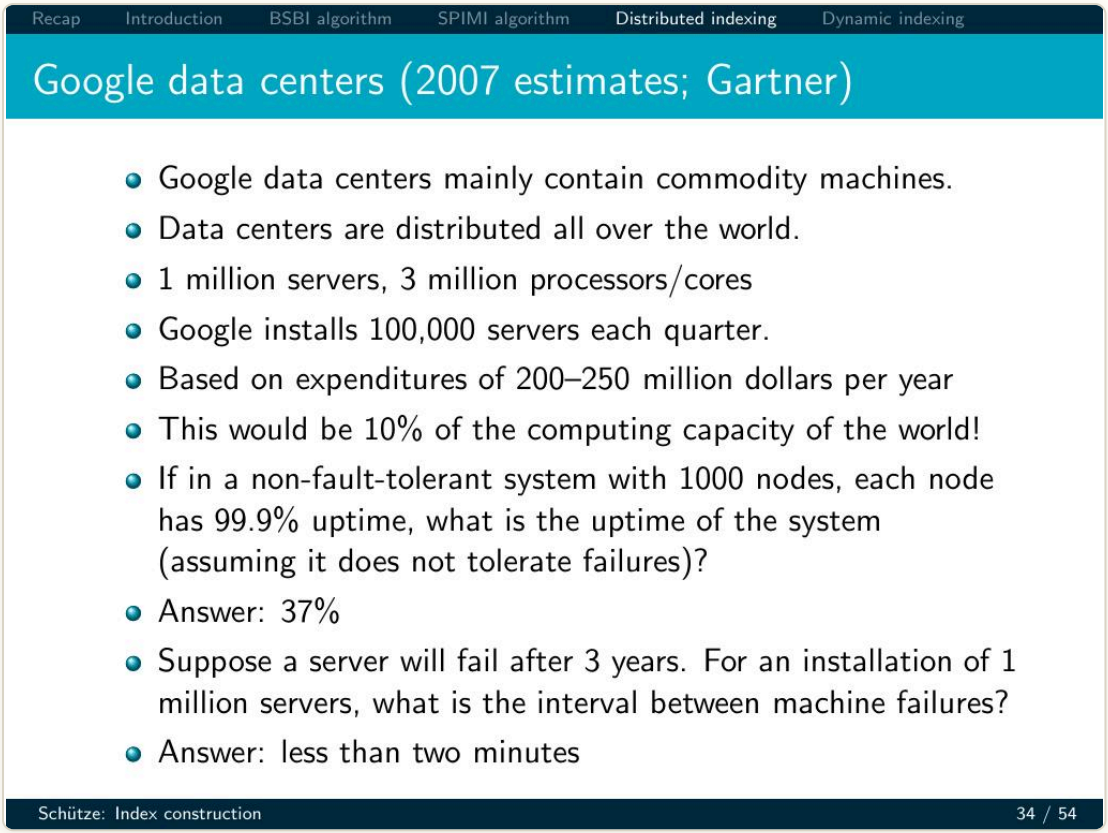


분산 색인

- 웹 규모 색인엔 **분산 컴퓨터 클러스터**가 필요
- 개별 기계는 **고장 나기 쉽다**
  - 예측 못 하게 느려지거나 멈춘다
- 이런 기계 풀을 어떻게 활용할까?

✍ 메모

— 넘어가도 되는 쪽

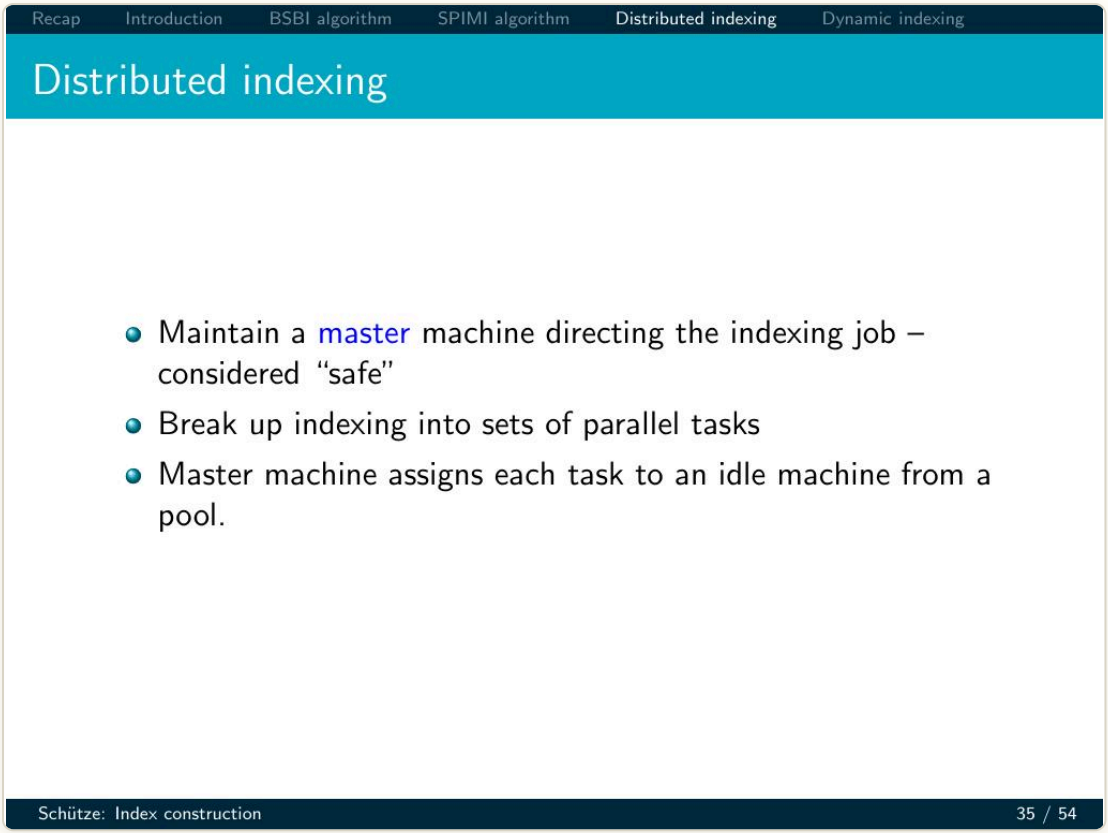


구글 데이터센터 (2007년 추정, 가트너)

- 주로 **범용(commodity) 기계**
- 데이터센터가 전 세계에 분산
- 서버 100만 대, 프로세서·코어 300만 개
- 분기마다 서버 10만 대 설치 (연간 2~2.5억 달러 지출 기준)
  - 전 세계 컴퓨팅 용량의 10%!
- 노드 1000개인 결함 비허용 시스템에서 각 노드 가동률이 99.9%면 시스템 가동률은? → **37%**
- 서버가 3년 뒤 고장 난다면 서버 100만 대에서 기계 고장 간격은? → **2분 미만**

✍ 메모

✏ 예·풀이 $0.999^{1000} \approx e^{-1} \approx 0.37$ / 3년 ≈ 9,460만 초 ÷ 100만 ≈ 95초

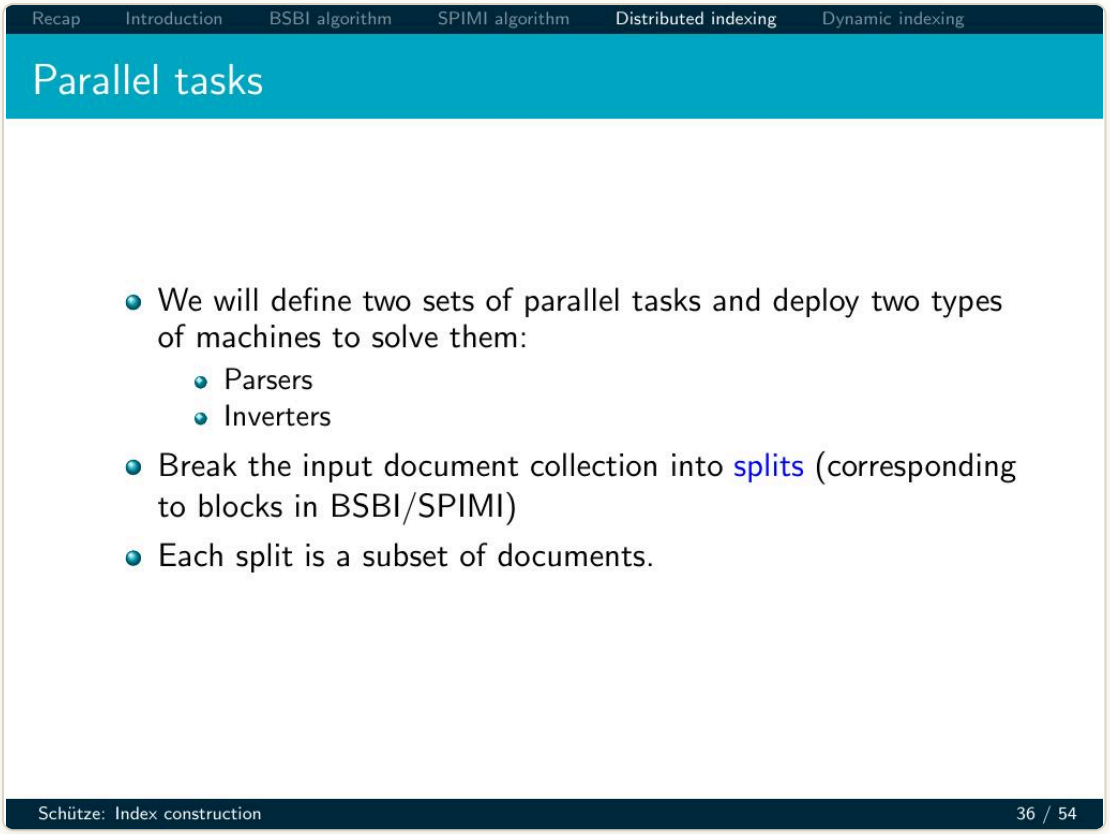


분산 색인 (계속)

- 색인 작업을 지휘하는 **마스터** 기계를 둔다 — '안전'하다고 간주
- 색인 작업을 **병렬 작업** 집합으로 나눈다
- 마스터가 각 작업을 풀의 **노는 기계**에 배정

✍ 메모

— 넘어가도 되는 쪽

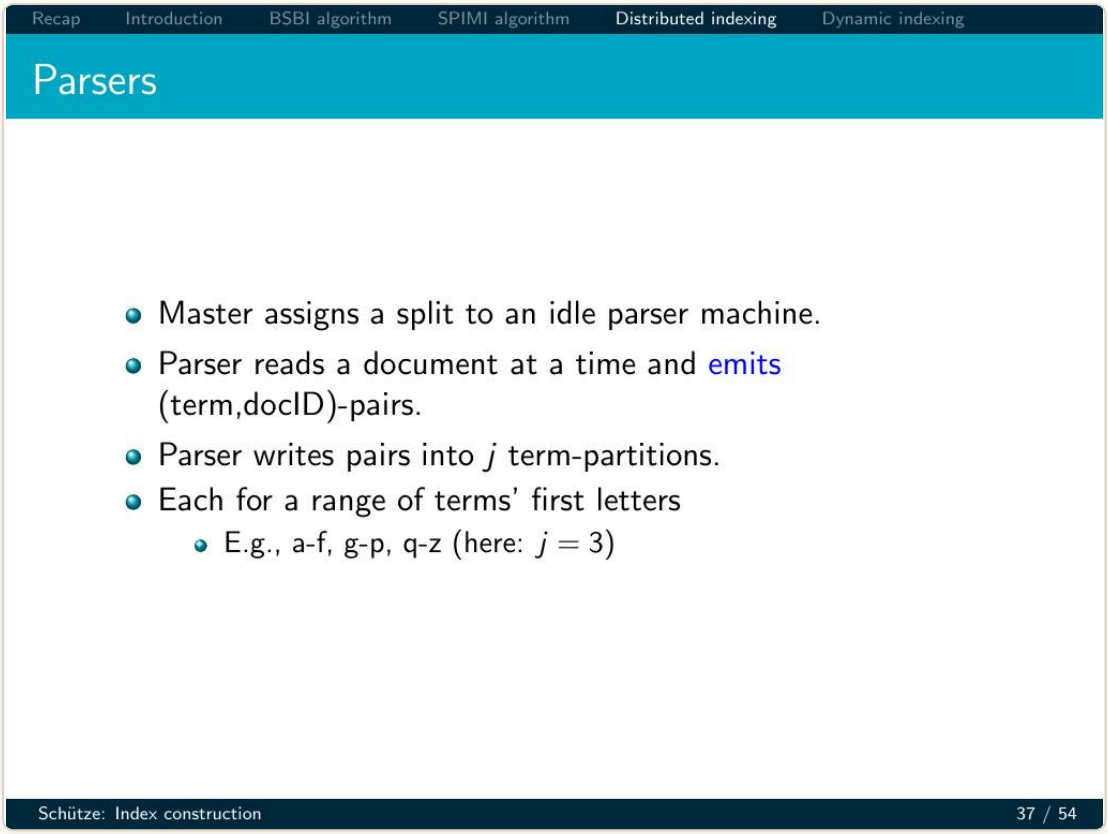


병렬 작업

- 두 종류의 병렬 작업, 두 종류의 기계
  - **파서(Parsers)**
  - **인버터(Inverters)**
- 입력 문서 컬렉션을 **스플릿(split)**으로 나눈다 (BSBI/SPIMI의 블록에 해당)
- 각 스플릿은 문서의 부분 집합

✍ 메모

— 넘어가도 되는 쪽

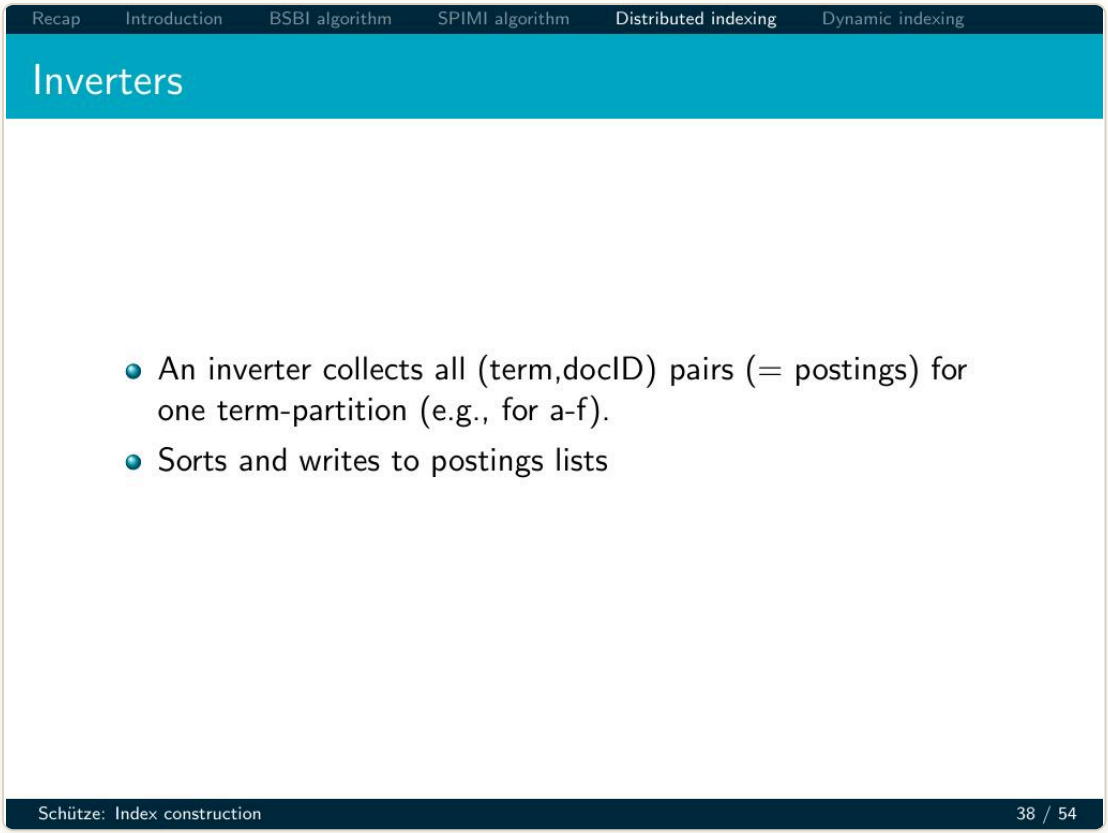


파서 (Parsers)

- 마스터가 스플릿을 노는 파서 기계에 배정
- 파서는 문서를 하나씩 읽어 **(term, docID) 쌍**을 내보냄
- 파서는 쌍을 **j개 단어 파티션**에 나눠 씀
  - 각각 단어 첫 글자의 범위
  - 예: **a-f, g-p, q-z** (여기선 j = 3)

✍ 메모

— 넘어가도 되는 쪽

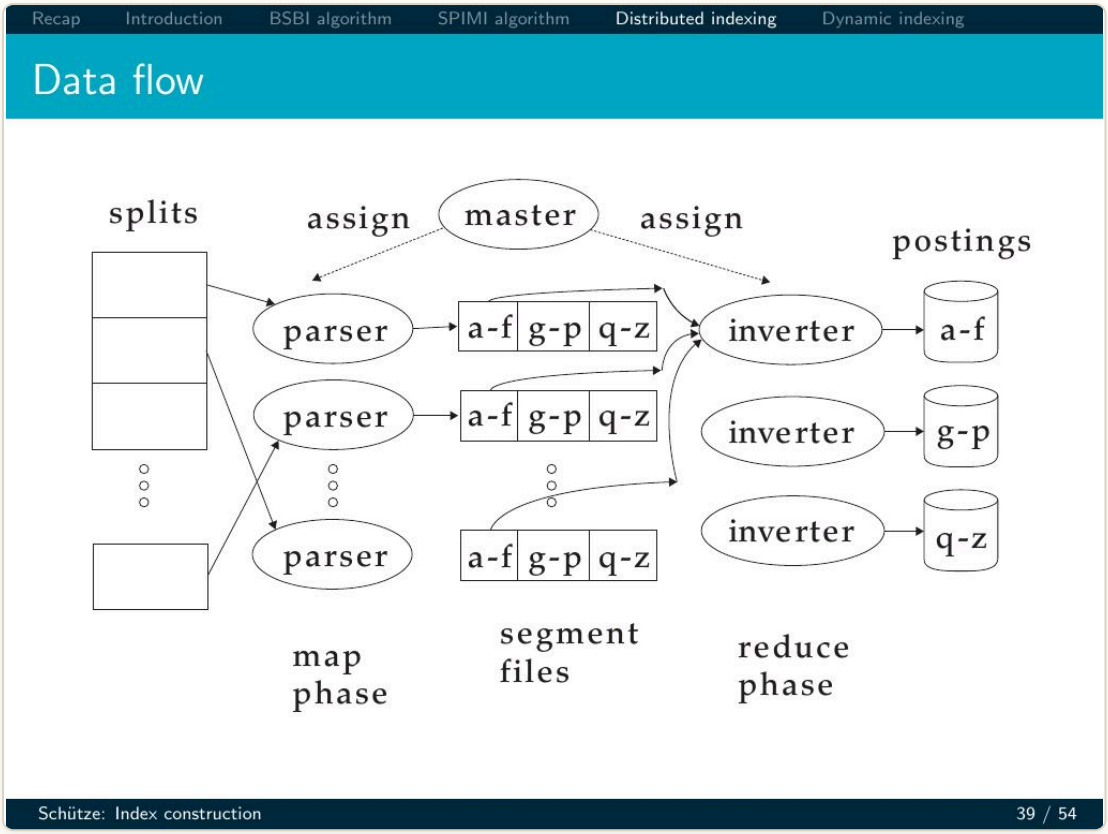


인버터 (Inverters)

- 인버터는 **한 단어 파티션**(예: a-f)의 모든 (term, docID) 쌍(= 포스팅)을 모은다
- **정렬**해서 포스팅 리스트로 쓴다

✍ 메모

★ 시험 파서 = map 단계(쌍 생성·분할) / 인버터 = reduce 단계(모아서 정렬)

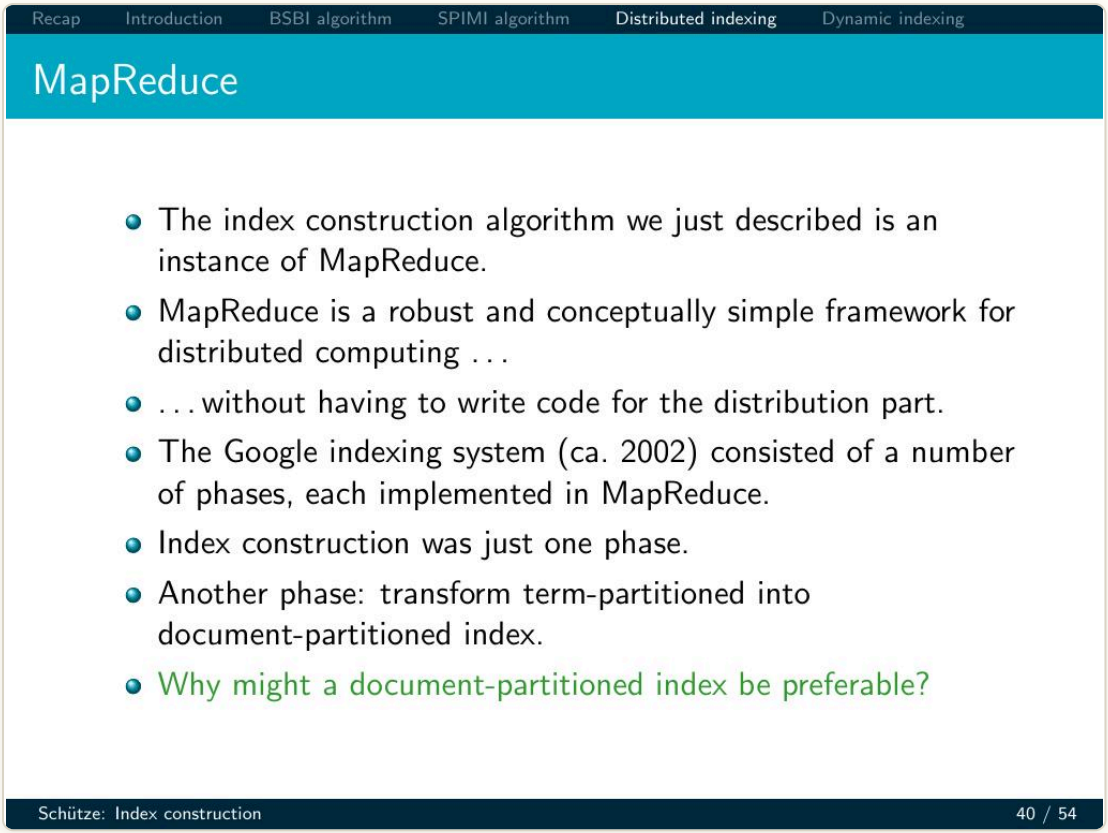


데이터 흐름 (그림)

- master가 splits를 parser에 배정(assign) → parser가 a-f / g-p / q-z **세그먼트 파일** 생성 (**map 단계**)
- master가 inverter에 배정 → 각 inverter가 한 범위의 세그먼트를 모아 **포스팅** 생성 (**reduce 단계**)

✍️ 메모

— 넘어가도 되는 쪽

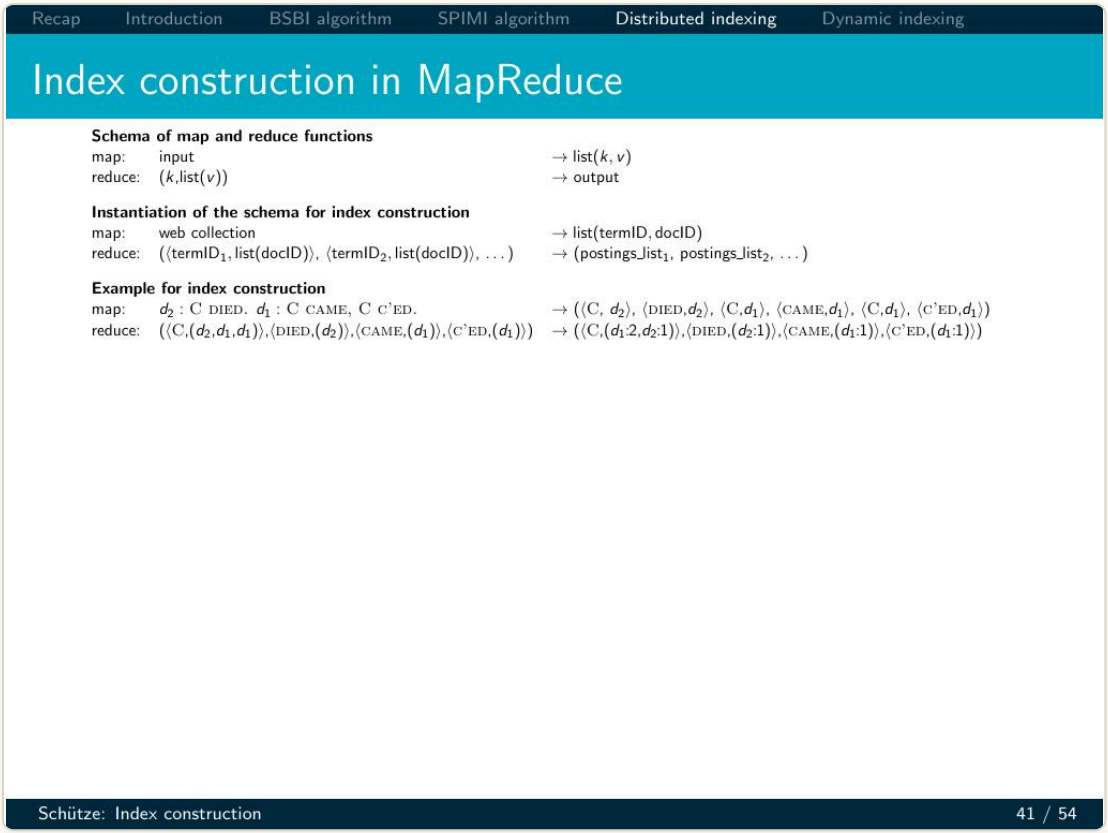


MapReduce

- 방금 본 색인 구축 알고리즘은 **MapReduce**의 한 예
- MapReduce = 분산 부분의 코드를 직접 짜지 않고도 쓸 수 있는 **견고하고 개념적으로 단순한** 분산 컴퓨팅 프레임워크
- 구글 색인 시스템(2002년경)은 여러 단계로 되어 있었고 각 단계가 MapReduce로 구현
  - 색인 구축은 그중 한 단계
  - 다른 단계: **단어 분할 색인 → 문서 분할 색인**으로 변환
- 왜 문서 분할 색인이 더 나을 수 있을까?

✍️ 메모

💡 이해 문서 분할: 질의를 모든 기계가 나눠 처리 → 부하 균형·한 기계 고장에도 일부 결과 가능

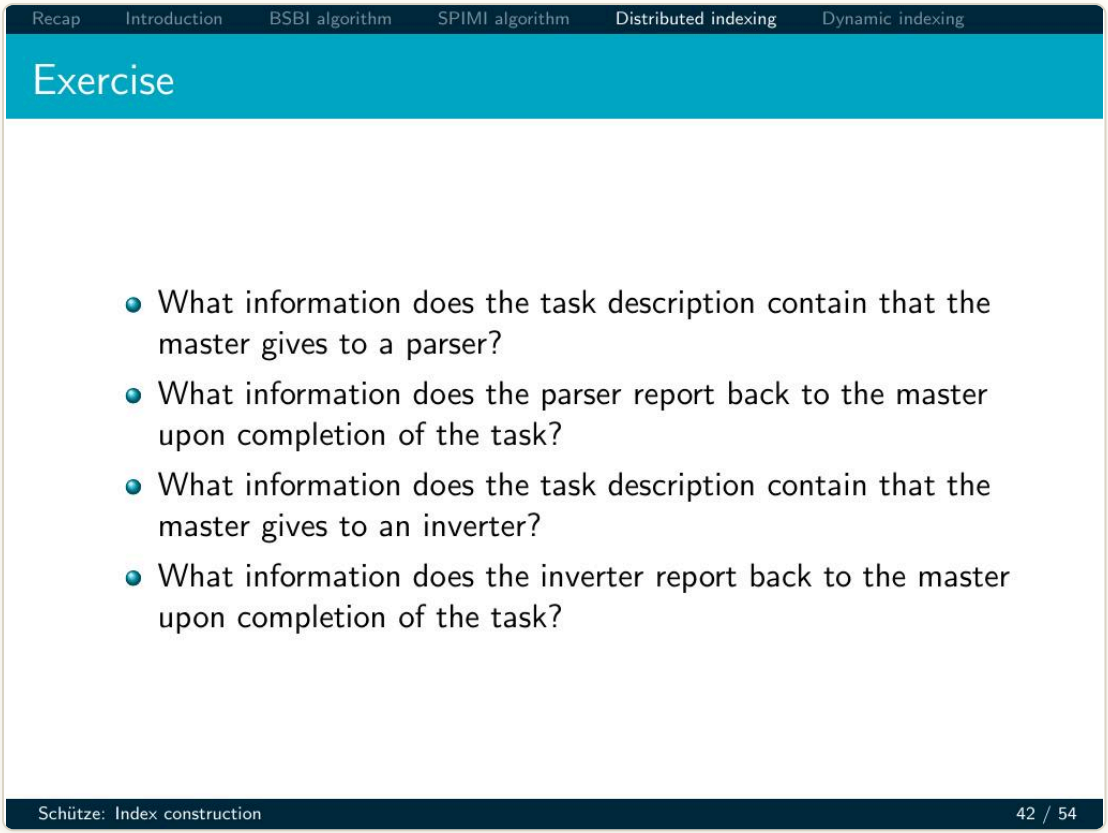


MapReduce로 색인 구축

- **스키마**: map: 입력 → list(k, v) / reduce: (k, list(v)) → 출력
- **색인 구축에 적용**: map: 웹 컬렉션 → list(termID, docID) / reduce: ⟨termID$_1$, list(docID)⟩, ... → (postings_list$_1$, postings_list$_2$, ...)
- **예**: map: d2: C died. d1: C came, C c'ed. → ⟨C, d2⟩, ⟨died, d2⟩, ⟨C, d1⟩, ⟨came, d1⟩, ⟨C, d1⟩, ⟨c'ed, d1⟩
  - reduce: ⟨C, (d2, d1, d1)⟩, ⟨died, (d2)⟩, ⟨came, (d1)⟩, ⟨c'ed, (d1)⟩ → ⟨C, (d1:2, d2:1)⟩, ⟨died, (d2:1)⟩, ⟨came, (d1:1)⟩, ⟨c'ed, (d1:1)⟩

✍ 메모

★ 시험 reduce 결과의 d1:2 = 문서 d1에 C가 2번 (단어 빈도)



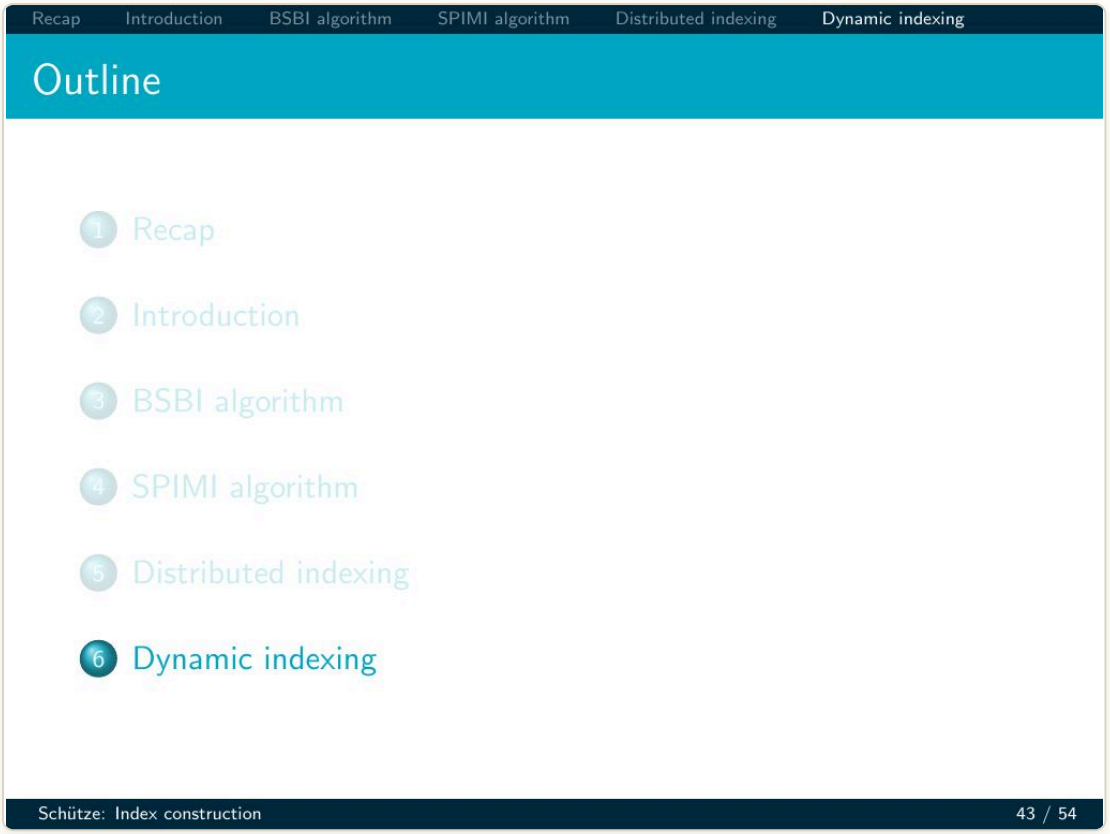
연습

- 마스터가 **파서**에게 주는 작업 설명엔 어떤 정보가 있나?
- 파서는 작업을 마치면 마스터에게 무엇을 보고하나?
- 마스터가 **인버터**에게 주는 작업 설명엔?
- 인버터는 마스터에게 무엇을 보고하나?

✍ 메모

✏ 예·풀이 파서: 받는 것 = 스플릿 위치 / 보고 = 만든 세그먼트 파일들의 위치

✏ 예·풀이 인버터: 받는 것 = 맡을 단어 파티션 + 세그먼트 파일 위치 / 보고 = 만든 포스팅 파일 위치

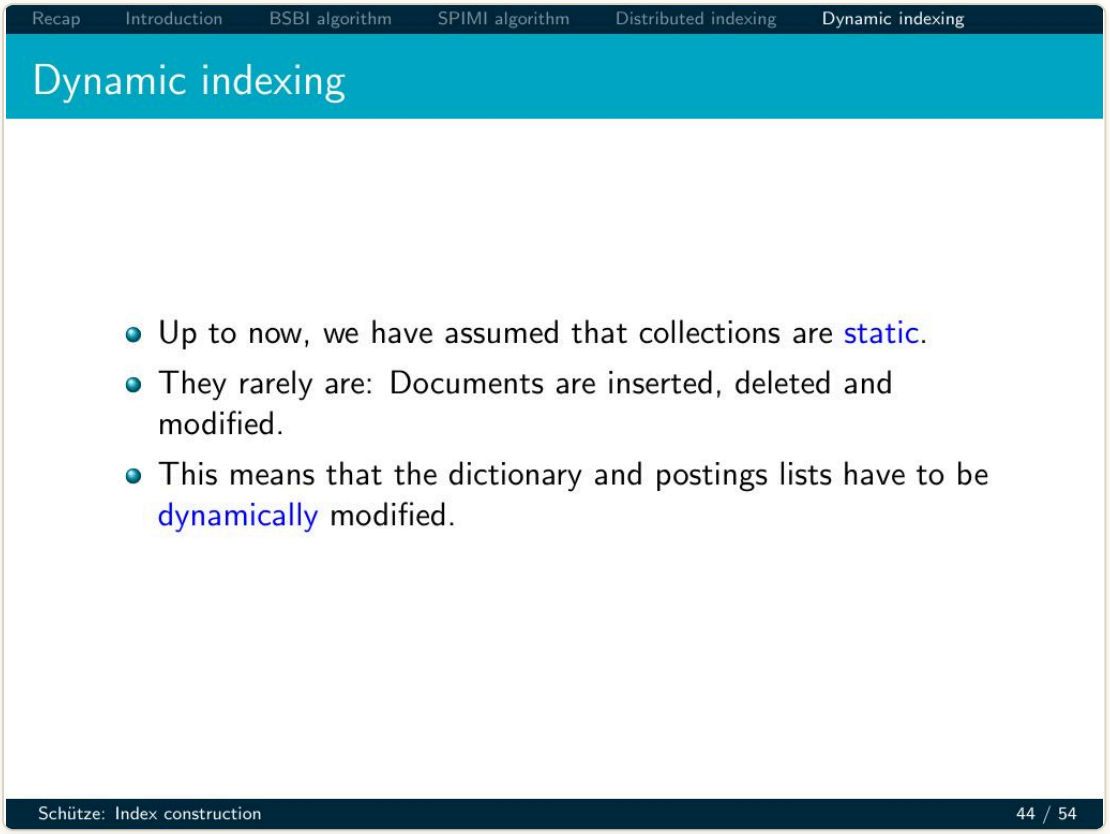


목차

- 복습 → 소개(하드웨어) → BSBI → SPIMI → 분산 색인 → 동적 색인
- 지금: **동적 색인**

✍️ 메모

— 넘어가도 되는 쪽

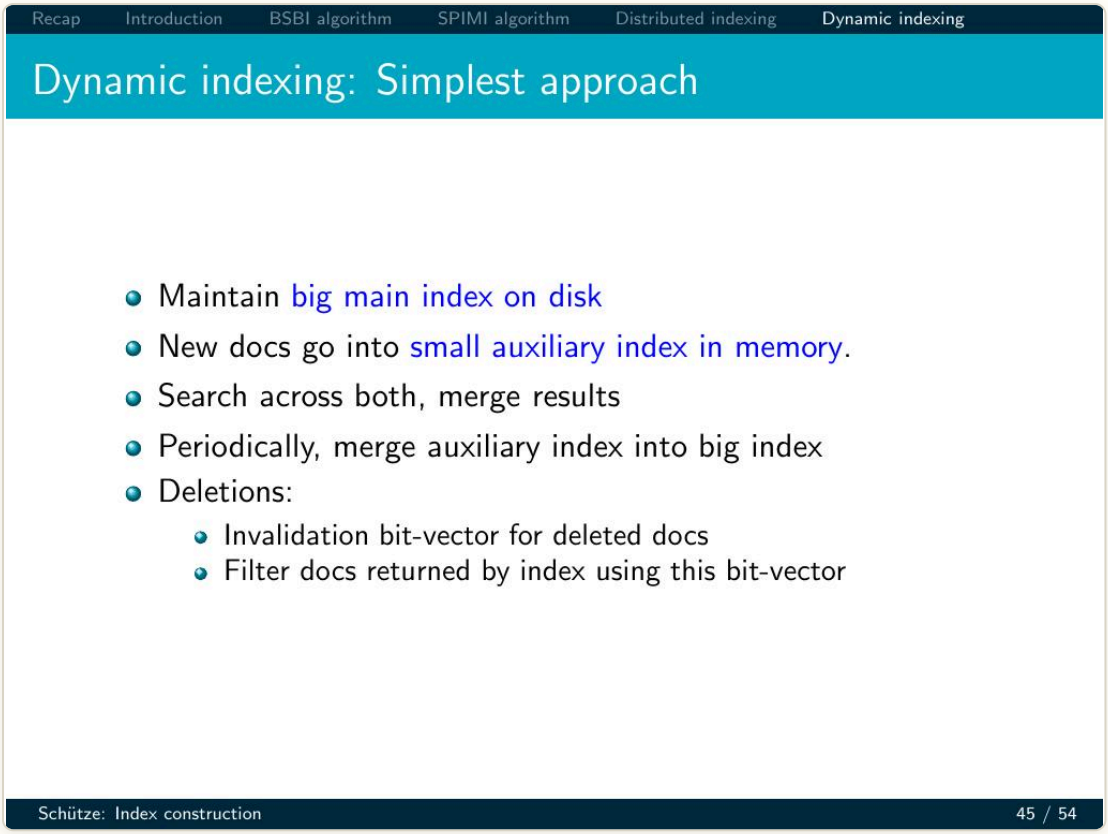


동적 색인 (Dynamic indexing)

- 지금까지 컬렉션이 **정적**이라고 가정했다
- 실제론 드물다: 문서가 **추가·삭제·수정**된다
- 그래서 사전과 포스팅 리스트도 **동적으로** 바뀌어야

✍ 메모

— 넘어가도 되는 쪽

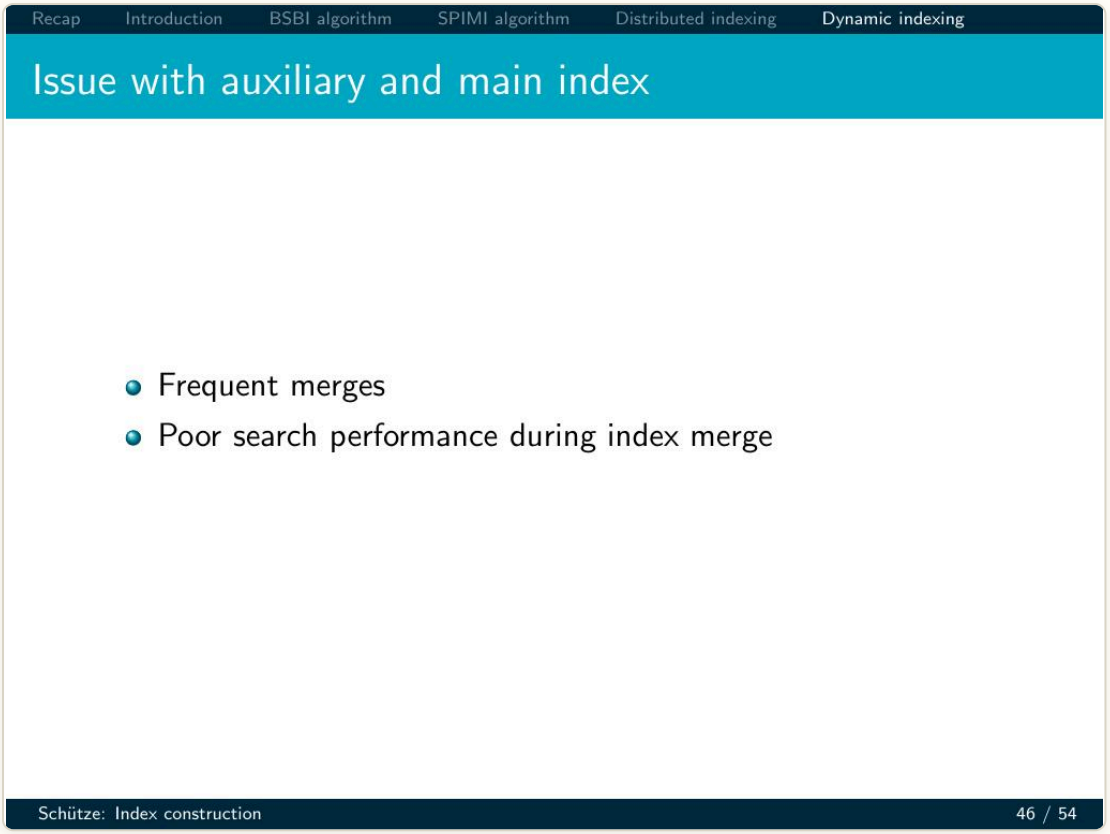


동적 색인: 가장 단순한 방법

- 디스크에 큰 **주 색인(main index)** 유지
- 새 문서는 메모리의 작은 **보조 색인(auxiliary index)**으로
- **둘 다** 검색해 결과를 합친다
- 주기적으로 보조 색인을 큰 색인에 **병합**
- 삭제
  - 삭제된 문서용 **무효화 비트 벡터(invalidation bit-vector)**
  - 색인이 반환한 문서를 이 비트 벡터로 걸러 냄

✍ 메모

★ 시험 주 색인 + 보조 색인 + 삭제 비트 벡터

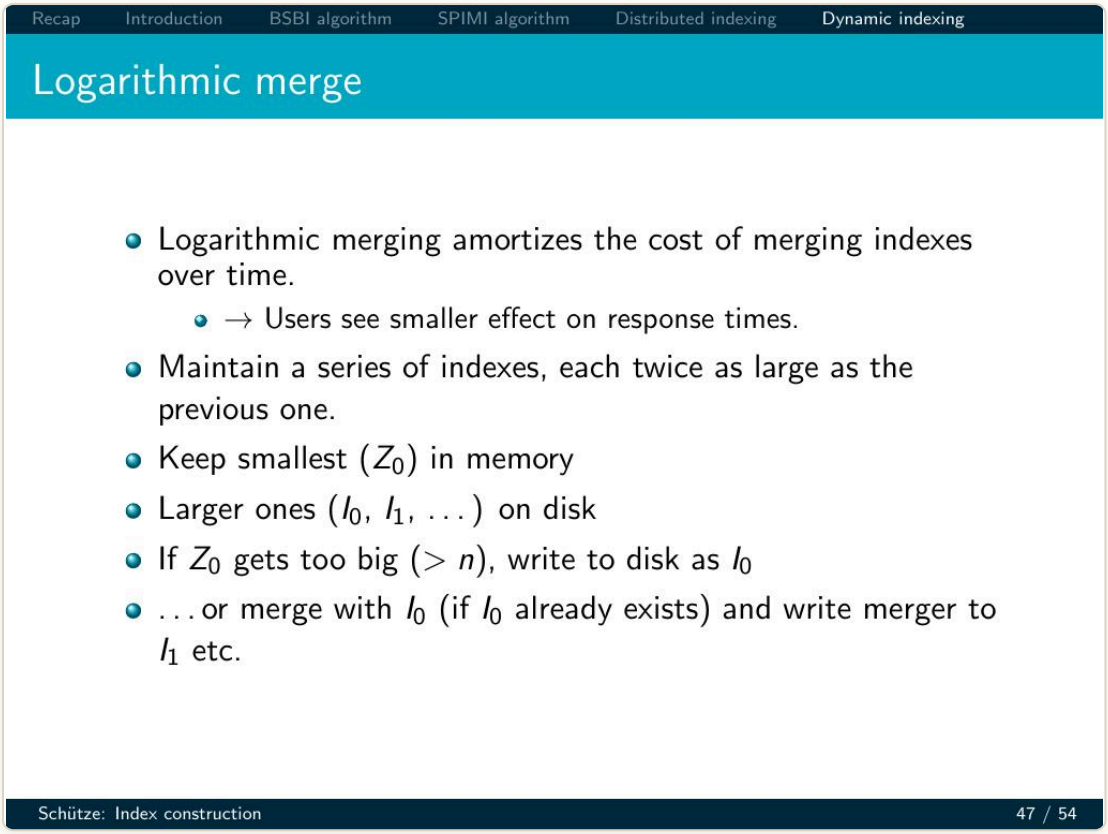


보조·주 색인 방식의 문제

- **잦은 병합**
- 색인 병합 중엔 **검색 성능이 나쁘다**

✍ 메모

— 넘어가도 되는 쪽

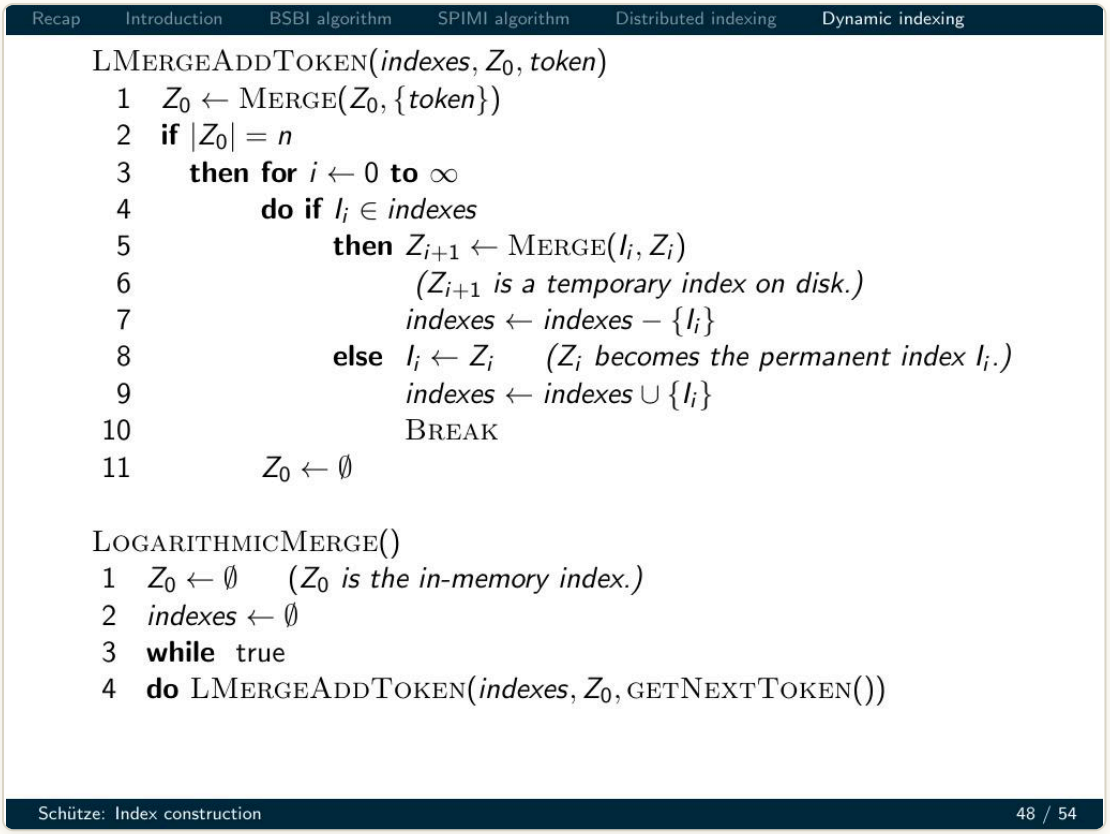


로그 병합 (Logarithmic merge)

- 로그 병합은 색인 병합 비용을 시간에 걸쳐 **분할 상환(amortize)**
  - → 사용자 응답 시간에 주는 영향이 작다
- 색인을 여러 개 유지 — 각각 **앞의 것의 두 배** 크기
- 가장 작은 것(**Z0**)은 메모리에
- 더 큰 것(**I0, I1, ...**)은 디스크에
- Z0가 너무 커지면(> n) 디스크에 I0로 쓴다
  - ... 또는 (I0가 이미 있으면) I0와 병합해 I1로 쓴다, 등등

✍ 메모

★ 시험 이진수 올림처럼 동작: 같은 크기 둘이 만나면 병합해 다음 단계로

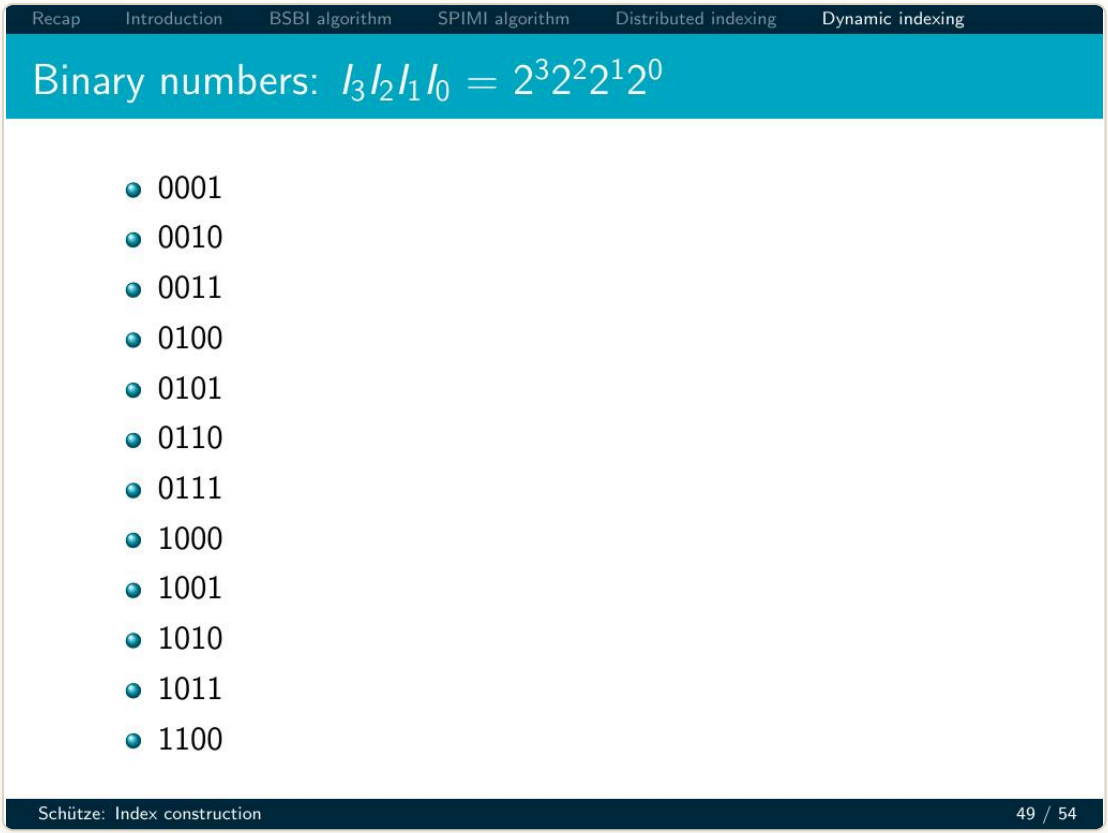


로그 병합 알고리즘

```
LMERGEADDTOKEN(indexes, Z0, token)
  Z0 ← MERGE(Z0, {token})
  if |Z0| = n:
    for i ← 0 to ∞:
      Ii가 있으면: Z(i+1) ← MERGE(Ii, Zi) (임시 디스크 색인), indexes에서 Ii 제거
      없으면: Ii ← Zi (영구 색인이 됨), indexes에 Ii 추가, BREAK
    Z0 ← ø
LOGARITHMICMERGE(): Z0 ← ø; indexes ← ø; while true: LMERGEADDTOKEN(indexes,
Z0, GETNEXTTOKEN())
```

✍ 메모

— 넘어가도 되는 쪽

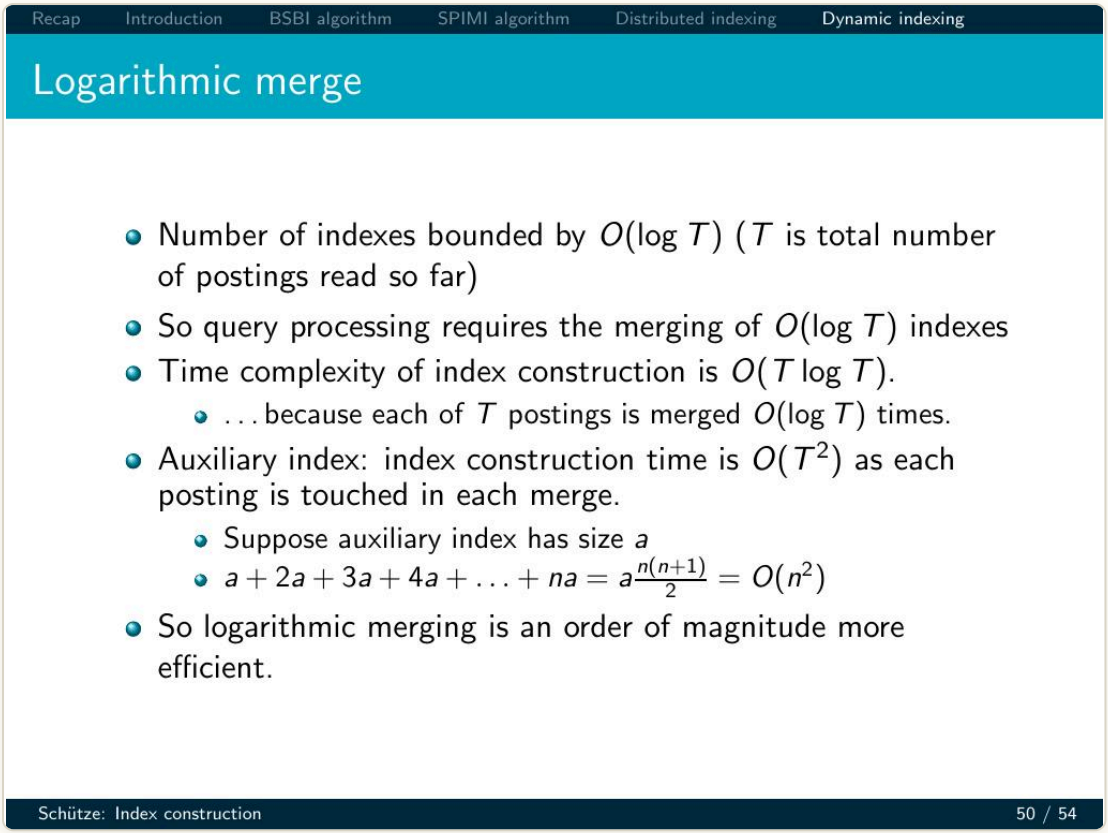


이진수: I3 I2 I1 I0 = 2^3 2^2 2^1 2^0

- 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001, 1010, 1011, 1100
- → 비트 i가 1 = 색인 Ii가 디스크에 있음. Z0가 찰 때마다 이진수에 1을 더하듯 올림(병합)

✍ 메모

✏ 예·풀이 0111 → 1000: I0·I1·I2가 연쇄 병합되어 I3 하나가 됨

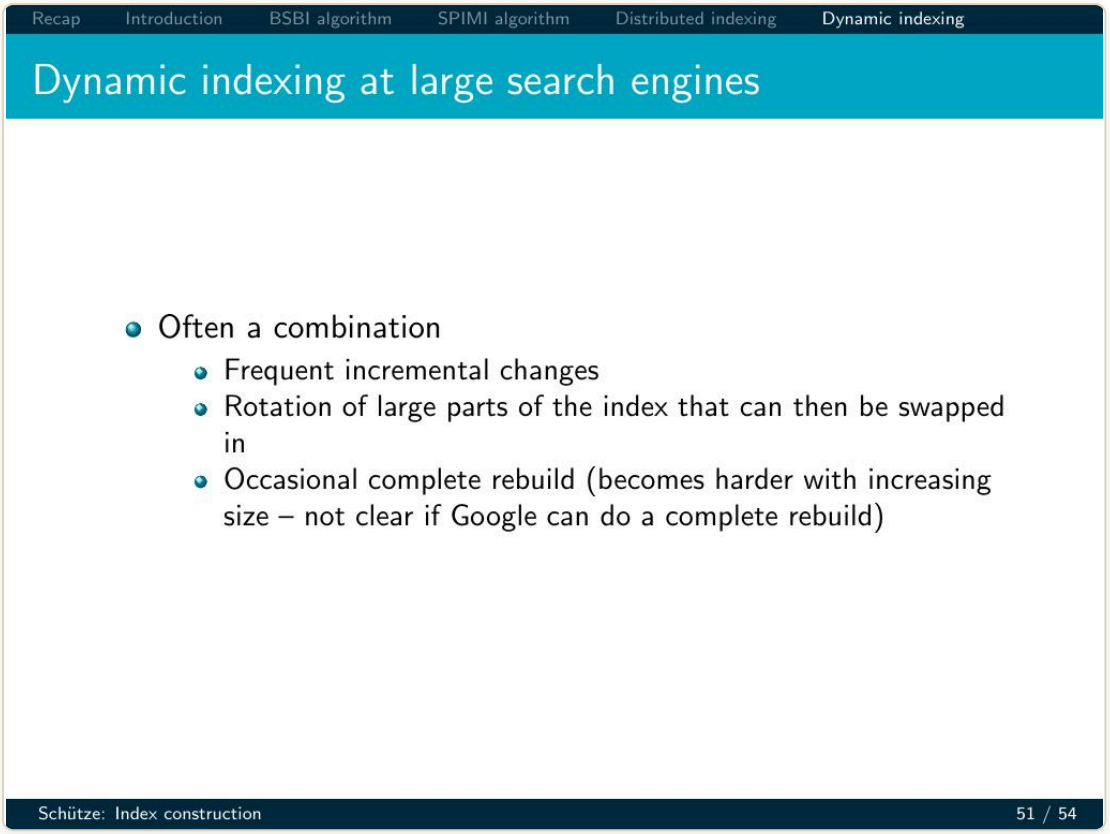


로그 병합: 복잡도

- 색인 개수는 **O(log T)** (T = 지금까지 읽은 포스팅 수)
- 그래서 질의 처리 때 **O(log T)개** 색인을 합쳐야
- 색인 구축 시간 복잡도 **O(T log T)**
  - ... 각 포스팅이 O(log T)번 병합되므로
- **보조 색인** 방식: 구축 시간 **O(T²)** — 병합할 때마다 모든 포스팅을 건드림
  - 보조 색인 크기가 a면: a + 2a + 3a + ... + na = a · n(n+1)/2
- 그래서 로그 병합이 **한 차원 더 효율적**

✍ 메모

★ 시험 보조 색인 O(T²) vs 로그 병합 O(T log T), 대신 질의 때 O(log T)개 색인 조회

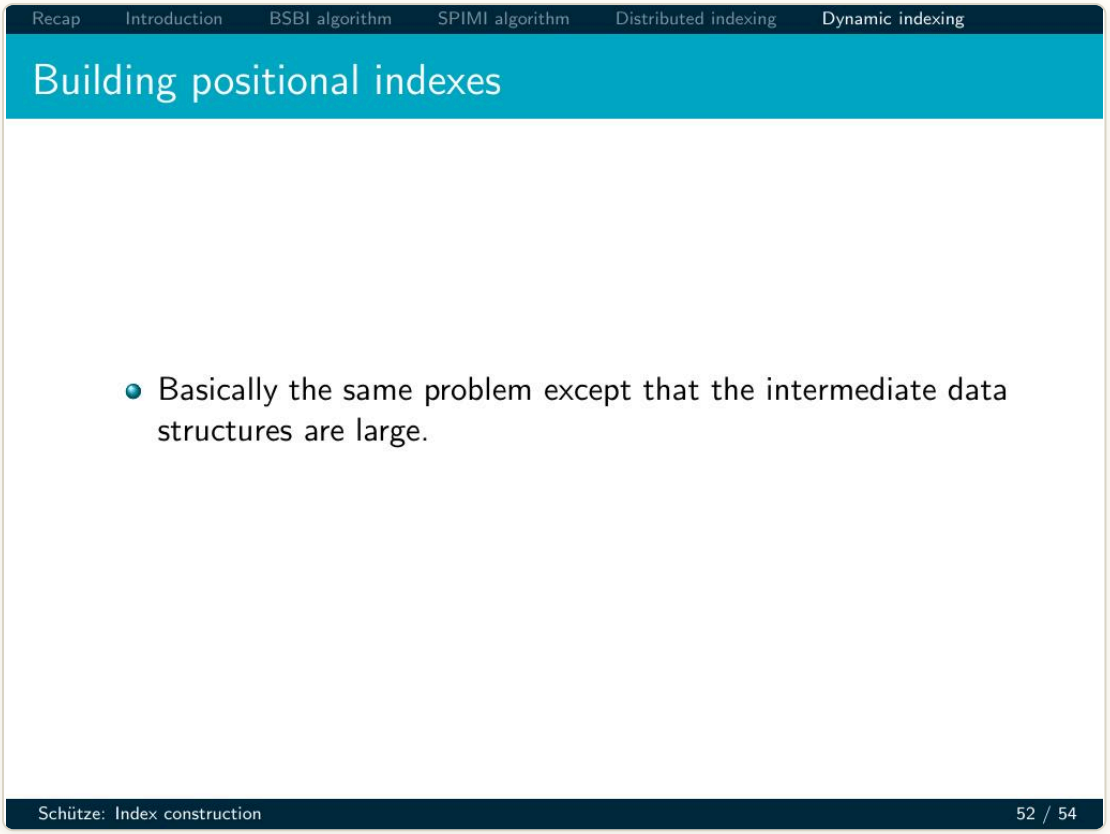


대형 검색 엔진의 동적 색인

- 보통 **여러 방법을 결합**
  - 잦은 **증분** 변경
  - 색인의 큰 부분을 **교체(rotation)** 해 바꿔 끼움
  - 가끔 **전체 재구축** (크기가 커질수록 어려움 — 구글이 전체 재구축을 할 수 있는지는 불분명)

✍ 메모

— 넘어가도 되는 쪽

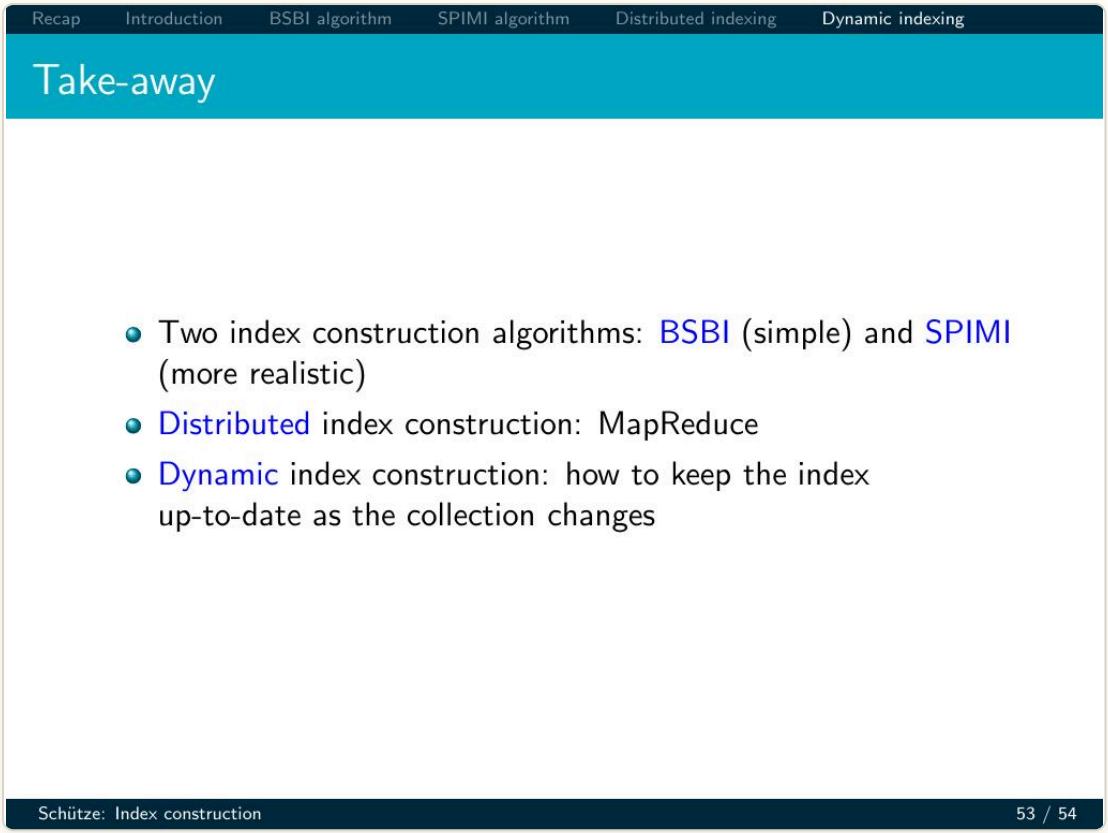


위치 색인 구축

- 기본적으로 같은 문제 — 다만 **중간 자료구조가 크다**

✍ 메모

— 넘어가도 되는 쪽

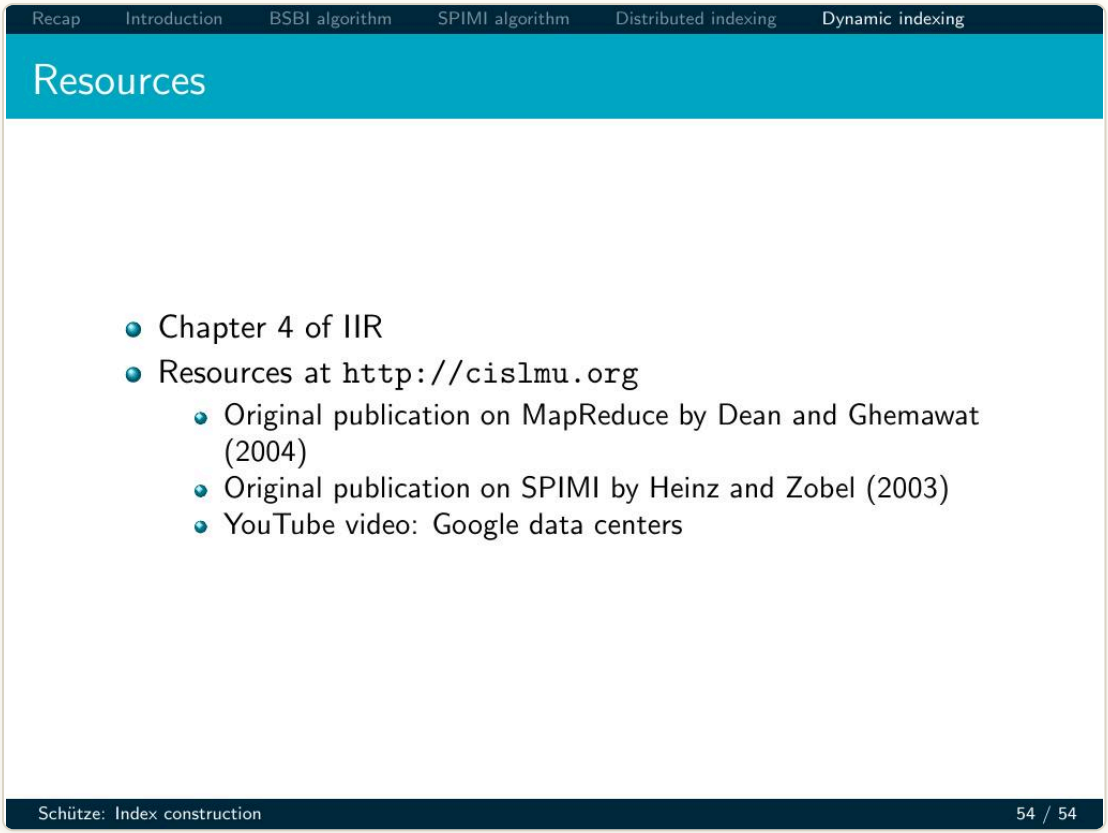


이번 강의에서 가져갈 것

- 색인 구축 알고리즘 두 가지: BSBI(단순)와 SPIMI(더 현실적)
- 분산 색인 구축: MapReduce
- 동적 색인 구축: 컬렉션이 바뀔 때 색인을 최신으로

✍️ 메모

— 넘어가도 되는 쪽



참고 자료

- IIR 4장, http://cislmu.org
- MapReduce 원 논문: Dean & Ghemawat (2004)
- SPIMI 원 논문: Heinz & Zobel (2003)
- 유튜브: 구글 데이터센터

✍ 메모

— 넘어가도 되는 쪽